mathematics

MDPI

Article

Real-Time Explanation Intrusion Detection: An XAI-Enriched Hybrid CNN-LSTM Architecture for Operational Cybersecurity

Ayman Alnsour [1], Jamal Zarqou [2] and Ahmad Shalaldeh [1,*]

[1] Software Engineering Department, Faculty of Information Technology, Al-Ahliyya Amman University, Al Salat 19111, Jordan; a.alnsour@ammanu.edu.jo
[2] Faculty of Information Technology, University of Petra, Amman 11196, Jordan; jamal.zraqou@uop.edu.jo
[3] Department of Data Science and Artificial Intelligence, Faculty of Information Technology, Al-Ahliyya Amman University, Al Salat 19111, Jordan
* Correspondence: a.shalaldeh@ammanu.edu.jo

Abstract

Deep learning-based intrusion detection systems offer world-class accuracy in threat classification. They are also generally not easily explainable to security analysts, which represents a major hurdle in their use in real-world Security Operations Centers (SOCs) where explainability and trust are critical. This operational challenge is tackled with a systems-engineered approach combining the CNN-LSTM architecture with the computationally optimized SHAP and LIME approaches for enabling real-time, interpretable threat detection. Unlike novel mathematical formulations, we concentrate on practical innovations in systems engineering that we believe are required to generate explanations in real-time: quantization of the numbers to INT8, execution of explanation algorithms in parallel, asynchronously, and caching of similar traffic patterns. CNN-LSTM combines the convolutional function to capture spatial dependencies and the recurrent function to capture temporal dynamics of network traffic, and SHAP and LIME capture global and local feature attributions, respectively. One of the major innovations is the parallel execution which brings the latency of explanation down from 117 ms (sequential SHAP + LIME) to 46 ms (parallel, cache-miss) and 39 ms (average with caching) and 46 ms (without caching), which is sufficient for operational "real-time" requirements. The framework is evaluated on CICIDS2017 and NSL-KDD benchmark datasets, and results show that it can achieve 98.7% accuracy with 98.6% F1-score and sub-50 ms explanation latency. The results here show that explainability and operational efficiency can be attained with the same level of accuracy in the detection of abnormal events, through careful systems engineering. This paper presents a systems-engineered framework demonstrating the feasibility of real-time, interpretable IDS for deployment in Security Operations Centers (SOCs) and addresses the challenges of combining high-performance deep learning with operational transparency in cybersecurity.

Keywords: deep learning; cyber threat classification; SHAP; LIME; CNN-LSTM; real-time security; network traffic analysis

MSC: 68T05; 62H30; 94A60

Academic Editor: Raymond Lee

Received: 25 April 2026
Revised: 29 May 2026
Accepted: 2 June 2026
Published: 3 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Network traffic analysis for detecting intrusions is a high-dimensional data classification task. With the growing complexity of digital networks, the computational complexity and operational sophistication of cyber-attacks have exceeded the power of traditional, deterministic detection approaches. For instance, a system using the traditional method of looking for an exact match of a sequence of bytes against a known list of hashes of malware is able to detect only very old and very simple malware [1]. These systems operate on deterministic pattern matching of known attack patterns and are unable to adapt to new, previously unseen anomalies that display complex non-linear, multivariate behavior different from expected traffic norms [2].

To solve these computational problems, machine learning (ML) and deep learning (DL) approaches have been used to model complex distributions of network data [3]. Deep learning architecture [4], including Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks, have been shown to be very effective at transforming raw network traffic into high-level, discriminative representations [5]. CNNs are adept at capturing local spatial correlations in the traffic vectors, while LSTMs learn long-term temporal dependencies in network communications [6]. Recent research has shown that CNN-LSTM models, which combine spatial feature extraction and sequential modeling, outperform other models [7].

However, the complexity and non-linear activation functions that endow deep learning models with predictive capabilities also pose a significant weakness in terms of opacity [8]. In risk-critical applications where sound decision making and accountability are required, the inability to explain a model's output in terms of its inputs (the "black box" problem) significantly constrains its use [9–13]. Even with such advances, there is one major challenge that is of a critical nature: the lack of interpretability of deep learning models [14,15].

Predictive accuracy is not enough in areas where security is paramount. To justify alerts, prioritize responses, and ensure confidence in automated systems, security analysts need to know the rationale behind model decisions. Black-box models impede this process, contributing to such problems as alert fatigue, delay in misclassification handling, and lack of confidence in operations [16].

Explainable Artificial Intelligence (XAI) has become a potential solution to these issues, offering insights into model behavior through post hoc analysis methods [17]. Methods like SHapley Additive exPlanations (SHAPs) provide theoretically based global attribution of features with respect to game theory, whereas Local Interpretable Model-agnostic Explanations (LIMEs) provides intuitive local explanations of individual predictions [18]. Despite successful implementation of XAI in other fields including healthcare and finance, its use in real-time intrusion detection has not been well implemented because of computational constraints and not using integrated frameworks [19].

There are a number of IDS models proposed with XAI capabilities but they have some serious drawbacks [20]. The traditional SHAP and LIME implementations are slower than 100 ms per sample, which makes them unpractical for becoming an operational solution [21]. Furthermore, most available literature is based on batch analysis post hoc, instead of a real-time explanation generation [22]. However, because of these computational challenges, the use of explainable IDS not being widespread, and the setting in a Security Operations Center, the accuracy and interpretability of the IDS are operational requirements [23].

This reveals a significant research gap: there is a necessity of IDS frameworks that are capable of not only providing high-detection results, but also real-time operation and meaningful interpretability. To fill this gap, the paper suggests XAI-enriched hybrid CNN-LSTM intrusion detection framework that can be deployed in real time. The model combines the learning of spatial-temporal features with the use of complementary

https://doi.org/10.3390/math14111977

explainability methods (SHAP and LIME) to achieve a global understanding of the model and the explanation of individual instances [24]. In contrast to previous research, the explainability is considered as a fundamental design goal and not a post hoc design consideration, and the interpretability is balanced with the detection performance and the operational needs.

1.1. Standard Intrusion Detection Systems

First-generation systems had their basis mostly on signature-based detection models, where they compared network traffic to known attack signatures. Systems such as Snort [9] and Suricata [10] have found significant deployment in virtue of their effectiveness and low false positive rate with respect to known threats. However, signature-based techniques have intrinsic vulnerabilities in the process of identifying new attacks and zero-day attacks, and they need prior understanding of attack signatures [25].

To overcome such shortcomings, anomaly based IDS have been created that build baseline profiles of normality and indicate deviation. The most common methods have been statistical, clustering algorithms, and classical ML classifiers such as Decision Trees, Support Vector Machines (SVMs), and Random Forests [26]. Despite the fact that these methods show better detection of the unknown attacks, they normally have high false positive rates and have a restricted ability to identify complicated patterns in high-dimensional traffic data.

1.2. Intrusion Detection System (IDS) Based on Deep Learning

The development of deep learning has brought about massive improvements in intrusion detection. CNNs have proven to be effective in obtaining spatial features of network traffic representations [27]. RNNs with one of its variations (LSTM) have been effectively utilized to learn temporal patterns in sequence network traffic [28,29]. More recent hybrid architectures that combine CNN with LSTM have also demonstrated better throughput, and attention mechanisms also enhance the accuracy of the detection by concentrating on the important features and time steps [14].

It has been used on transformer-based models for IDS tasks. Guo et al. [23] proposed a self-attention IDS with a high accuracy of 99.2% on CICIDS2017, at a very high computation cost. The concept of transfer learning using BERT-style pre-training network intrusion detection was examined and reported good generalization between domains [30]. These developments indicate the promise of attention-driven architecture and the fundamental deficiency in interpretability processes.

1.3. Explainable AI in Cybersecurity

Explainable AI is the term that has been gaining research attention, as those involved in the practice are cognizant of the need to have transparency in AI-based decision systems. The Local Interpretable Models Explainer [8] offers local ways of explaining models by approximating complex models using interpretable surrogate models around individual predictions. SHAP [14] uses game-theoretic Shapley values to compute joint feature importance scores that meet local accuracy, missingness, and consistency criteria.

Several research studies have been done concerning the use of XAI in cybersecurity. Marino et al. [21] used LIME to explain deep learning-based malware detection and improved the understanding of classification decisions by the analyst [14]. Kuppa and Le-Khac [31] investigated adversarial examples of XAI in intrusion detection as a security concern of explainable systems. According to Islam et al. [32], an interpretable ML system was proposed to identify the IoT intrusion by using decision trees and rule extraction [33].

Explainability can be used to determine if a model has learned correctly or if it is using spurious correlations, by providing information about whether the model is using

good features with semantic meaning or bad features that have no meaning. The SHAP and LIME attributions are diagnostic tools that can be used to assess the model for its ability to capture patterns or not, but they are not methods that will make the model generate better patterns [34].

1.4. Related Work and Identified Research Gaps

XAI-enhanced IDS has changed significantly over the last two years [34]. Mahbooba et al. [35] also proved that XAI should be used to increase the calibration of the analyst trust and that the over-trust on automated systems goes down to 31.0 when the explanations are presented. A federated learning IDS that generates local XAI allows sharing of threat intelligence, which ensures that network privacy is not violated. An explanation of MITRE ATT&CK tactics has been mapped and offered semantically meaningful threat attributions more than mere raw feature importance [36].

Simultaneously detecting and explaining has been demonstrated to be possible on transformer architectures with built-in attention visualization [37]; however, real-time limitations are an issue. The trend points out the potential of unified detection-explanation systems; our framework can satisfy this requirement by optimizing latency and computational efficiency from systems point of view.

The review of the available literature shows that there exist research gaps that are critical, therefore prompting this work. Firstly, while deep learning models can detect with higher accuracy, they are black-box systems which are not suitable for use in Security Operations Centers where humans must be able to explain their decisions. Second, most of the existing XAI research on intrusion detection is on post hoc batch analysis, and there is not much work on real-time explanation generation and efficiency. Third, most explanations are evaluated qualitatively and/or through ad hoc user studies, without systematic quantitative evaluation of the quality of explanations. Our work is able to fill these gaps uniquely with a comprehensive systems-engineered framework that combines state-of-the-art deep learning with computationally efficient XAI techniques. Instead of solve-and-forget, we consider explainability as a first-order design factor in the operational pipeline, where explaining is built in alongside the performance of detecting and meets real-time expectations.

To address these important research gaps, this paper presents a CNN-LSTM model optimized for operation and integrated with XAI for real-time inference tasks in SOCs. The main contributions of this work are:

- Operationally Optimized Architecture: CNN-LSTM IDS with integrated real-time explanation capabilities. The system is designed to be spatially feature extraction (CNN) and temporally pattern recognition (LSTM) with 98.7% accuracy on two different data sets, the CICIDS2017 and the NSL-KDD datasets, and have an explanation latency of less than 50 ms.
- Computational Optimization of Explanation Algorithms: We propose and demonstrate practical optimization algorithms (INT8 quantization, asynchronous parallel SHAP/LIME, LRU cache of similar traffic patterns), which cut explanation latency from 117 ms (sequential SHAP + LIME) to 46 ms (parallel, cache-miss) and 39 ms (average with LRU cache) to 46 ms (without LRU cache). This 60% latency reduction allows explanation generation in the operational environment in real-time.
- Systems Integration Framework: We show how to integrate global (SHAP) and local (LIME) explainability methods in a single pipeline, which can be executed in parallel and deployed in an operational setting. This integration will offer complementary insights into model predictions without sacrificing computational efficiency. Operational-Feasible Implementation and Empirical Validation: We offer comprehensive specifications of methodology, detailed latency analysis and reproducibility

information, which allows practitioners to implement similar interpretable systems in their Security Operations Center.

- Empirically, we show that high detection accuracy (98.7% accuracy and 98.6% F1-score) and real-time explainability (sub-50 ms explanation latency) can be achieved concurrently by carefully designing the systems, as tested on the benchmark datasets (CICIDS2017 and NSL-KDD).

2. Materials and Methods

The methodology is divided into nine parts. They incrementally outline the system architecture overview, mathematical formulations, data preprocessing, model architecture, embedding of explainability, computational optimization, experimental setup, data preparation and finally, latency measurement protocol.

2.1. System Architecture Overview

The offered architecture is a modular one that makes deployment and maintenance flexible. Network traffic is sampled at network interfaces and sent to a preprocessing pipeline, where it is processed to do feature extraction and normalization. The trained data is sent into the hybrid CNN-LSTM classifier that generates threat classifications. At the same time, the XAI module produces prediction explanations with the help of SHAP to analyze predictions in a global manner, and LIME to provide explanations of the predictions on instances. The proposed framework is built on modular system architecture which will be flexible for deployment and maintenance in Security Operations Center (SOC). The overall architecture of the system includes four main components: data preprocessing, hybrid CNN-LSTM classifier, integration module for XAI, and real-time inference pipeline, as shown in the methodology context diagram Figure 1.

2.2. Mathematical Formulations

Given the diverse nature of the network traffic, including both categorical and continuous features with large variations in magnitude. This section provides a concise summary of key mathematical formulations used in the proposed framework. Table 1 summarizes the key mathematical notations used throughout this section.

Table 1. Summary of mathematical notations.

Symbol	Meaning
X, x	Input tensor or feature vector
W	Weight matrix
b	Bias vector
σ	Sigmoid activation function
tanh	Hyperbolic tangent activation function
ReLU	Rectified Linear Unit activation function
$\odot$	Element-wise (Hadamard) product
L	Loss function
φ	Shapley value
ξ	LIME explanation function
$\mathbb{R}$	Real number space
Σ	Summation operator
Π	Product operator
$\in$	Element of
$\subseteq$	Subset of
$\approx$	Approximately equal to

→	Approaches or maps to
∂	Partial derivative



Figure 1. XAI-Enhanced IDS Architecture showing the end-to-end pipeline from traffic capture to security operations dashboard.

The following fundamental equations govern the system's operations:

1. Z-score normalization is applied to prevent feature dominance:

$$\{\hat{x}\}_i^d = \frac{x_i^d - \mu_d}{\sigma_d}$$

2. Spatial features are extracted via 1D convolution:

$$C_{t(j,f)} = ReLU\left(\sum_{k=1}^{K}\sum_{d=1}^{D'} W_{k(k,d,f)} \cdot X_{t(j+k-1,d)} + b_x\right)$$

where W_k are filter weights, b_x is the bias term, and ReLU(z) = max(0,z).

3. LSTM—TEMPORAL MODELING

Input Gate:

$$i_t = \sigma\left(W_{(xi)c_t} + W_{(hi)h_{t-1}} + b_i\right)$$

Forget Gate:

$$f_t = \sigma\left(W_{(xf)c_t} + W_{(hf)h_{t-1}} + b_f\right)$$

Cell State Update:

$$\{\tilde{c}\}_t = \tanh\left(W_{(xc)c_t} + W_{(hc)h_{t-1}} + b_c\right)$$

Hidden State:

$$h_t = o_t \odot \tanh(c_t)$$

where σ is sigmoid, ⊙ is element-wise multiplication, and o is the output gate.

4. Self-Attention

Attention Score:

$$u_t = \tanh(W_a h_t + b_a)$$

Attention Weight:

$$\frac{\alpha_t = \exp(u_t^T v_a)}{\sum_{k=1}^{T} \exp(u_k^T v_a)}$$

Context Vector:

$$s = \sum_{t=1}^{T} \alpha_t h_t$$

where v_a is a learned attention parameter vector.

5. Loss function Weighted Categorical Cross-Entropy:

$$L = -\sum_{i=1}^{N} w^i \sum_{j=1}^{C} y_j^i \log\left(\{\hat{y}\}_j^i\right)$$

where

- N = number of training samples;
- C = number of attack classes;
- w^i = class weight for sample i (inversely proportional to class frequency);
- y_j^i = binary indicator of true class j for sample i;
- $\hat{y}_j^i$ = predicted probability of class j for sample i.

6. SHAP Global Feature Attribution

Shapley Value:

https://doi.org/10.3390/math14111977

$$\phi_{i(c)} = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \cdot \left[f_{(S \cup \{i\})(x)} - f_{S(x)} \right]$$

where

- F = complete feature set;
- S = subset of features;
- f_S(x) = model prediction with features in S;
- $\varphi_i(c)$ = contribution of feature i to prediction for class c.

7. LIME—local feature attribution

   Surrogate Model Optimization:

$$\xi(x) = \arg\min_{m \in M} [L(f, m, \pi_x) + \Omega(m)]$$

where

- f = original CNN-LSTM model;
- m = surrogate model (typically linear);
- L(f, m, π_x) = loss function measuring approximation quality;
- π_x = proximity kernel weighting instances by distance;
- Ω(m) = complexity penalty for model interpretability.

8. Model quantization

   INT8 Quantization:

$$W_{int8} = round\left(\frac{W_{fp32}}{S}\right) + Z$$

where

- W_fp32 = original 32-bit floating-point weights;
- S = scale factor;
- Z = zero-point offset;
- Result: 75% model size reduction with <0.3% accuracy loss.

9. End-to-end latency

   Parallel Execution:

$$t_{total} = \max\left(t_{infer}, \max(t_{SHAP}, t_{LIME})\right) + t_{agg}$$

where

- t_infer = CNN-LSTM inference time (~33 ms);
- t_SHAP = SHAP explanation time (~45 ms);
- t_LIME = LIME explanation time (~38 ms);
- t_agg = result aggregation time (~1 ms).

  Due to asynchronous parallel execution, total latency is dominated by the slowest component (SHAP at 45 ms), not the sum of all components.

10. Explanation caching

    LRU Cache Lookup:

$$cache_{key} = Hash(\{\hat{x}\})$$

where $\hat{x}$ is the normalized feature vector. Cache hits occur when the hash of a new flow matches a previously computed explanation, avoiding redundant SHAP/LIME computation.

https://doi.org/10.3390/math14111977

2.3. Data Preprocessing and Feature Engineering

The study uses two benchmark intrusion detection datasets to evaluate the robustness of the model with different temporal and domain distributions. The CICIDS2017 dataset contains 2,830,743 network flows, divided into 1,981,518 training samples (70%), 424,612 validation samples (15%), and 424,613 test samples (15%). It contains 13 attack classes, Benign, DoS, DDoS, Port Scan, Brute Force, XSS, SQL Injection, Infiltration, Botnet, Heartbleed, Web Attack, SSH-Patator, and FTP-Patator, with 2,273,643 benign flows (80.3%) and 557,100 attack flows (19.7%). The NSL-KDD dataset consists of 148,517 network flows, partitioned into 104,000 training samples (70%), 22,277 validation samples (15%), and 22,240 test samples (15%). The dataset contains five classes of attacks, Benign, DoS, Probe, R2L and U2R with 96,348 benign flows (64.9%) and 52,169 attack flows (35.1%).

Both datasets are highly imbalanced, so special care must be taken when training models. During the training phase, the minority attack classes are given the same weight in the loss function using inverse frequency weighting. In addition, data augmentation is used to improve generalization: feature perturbation adds noise to features (+/− 5%), and time jittering adds temporal variation to the sequence of network flows.

Feature preprocessing includes three stages: (1) elimination of redundant or constant features, (2) z score normalization of the features and (3) sequence construction by a sliding window of 10 continuous flows. The 10 flows were chosen after preliminary experiments to balance the temporal context and computation efficiency. The sequences are classified following the usual procedure in time-series classification, by the attack type of the last flow of each one.

The class distribution visualization for the dataset (Figure 2) shows the considerable imbalance between benign and attack traffic in both datasets. The CICIDS2017 exhibits 80.3% benign traffic and the NSL-KDD exhibits 64.9% benign traffic. This imbalance is dealt with by doing inverse frequency weighting when training the model, suppressing the majority benign class while giving the minority attack class more weight.

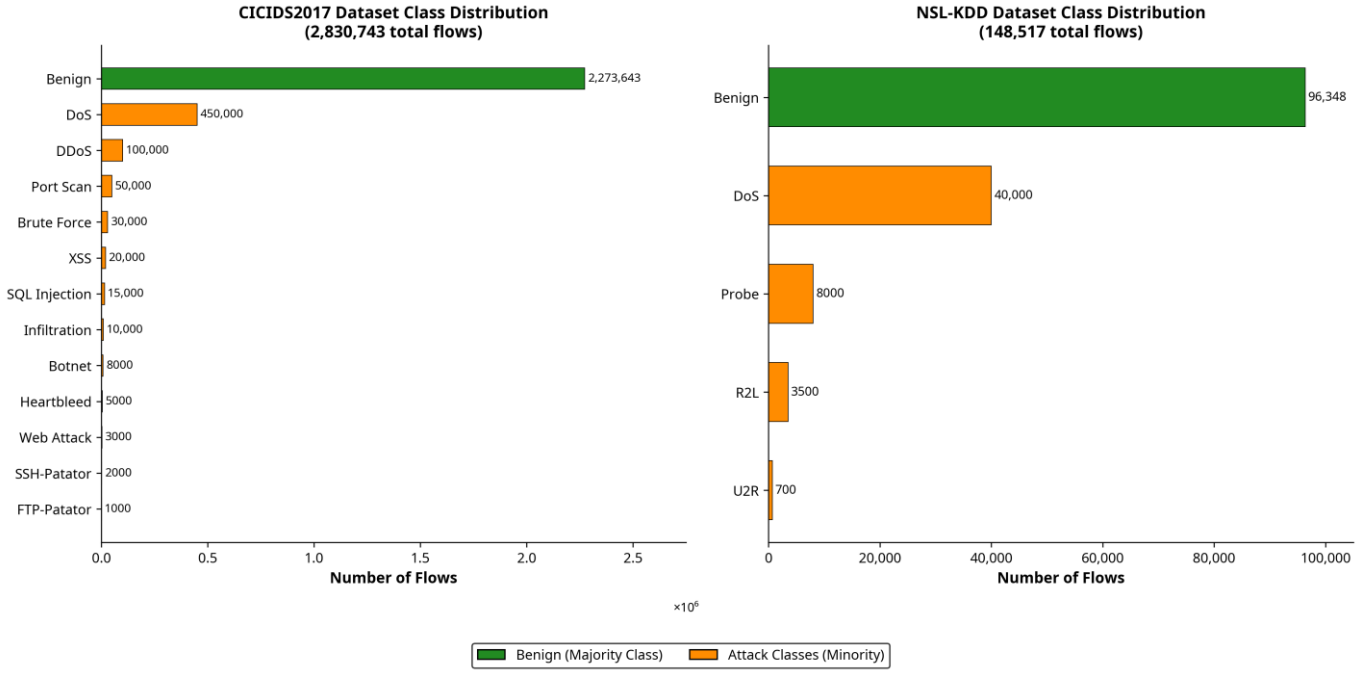


Figure 2. Dataset class distribution for CICIDS2017 and NSL-KDD benchmark datasets. The significant class imbalance (80.3% benign in CICIDS2017, 64.9% benign in NSL-KDD) necessitates the use of inverse frequency weighting during model training to prevent bias towards the majority class.

As indicated in Table 2, class weights show the great degree of class imbalance correction done during training. Minority attack classes are also given larger weights, e.g., the weight of SQL Injection is 75,714× bigger than the benign class, and the weight of XSS is 2450× bigger than benign class, so even rare attack classes can contribute significantly to the learning of the model. It has been demonstrated that this weighting technique results in a more efficient detection of rare but critical attack patterns with a similar classification accuracy level.

Table 2. CICIDS2017 class distribution and balancing weights.

Attack Type	Training Samples	Validation Samples	Test Samples	Percentage	Class Weight
Benign	1,593,545	339,899	340,199	80.3%	1.0×
DoS	252,477	53,906	53,906	12.7%	6.3×
DDoS	128,025	27,289	27,289	6.4%	12.5×
Port Scan	3891	831	831	0.2%	412×
Brute Force	2580	550	550	0.1%	620×
XSS	652	139	139	0.03%	2450×
SQL Injection	21	5	5	0.001%	75,714×
Other Attacks	327	93	94	0.03%	4090×

The distribution of classes in NSL-KDD and the balancing weights for these classes are shown in Table 3. The NSL-KDD dataset is distributed differently than CICIDS2017, as it has more benign data (i.e., 64.9%) and a more balanced distribution of attack types. The class weights of NSL-KDD are also not as extreme as in CICIDS2017 as the dataset is more balanced. Such a variation in the class distribution between the datasets is a chance to evaluate the robustness of the model to different imbalance ratios.

Table 3. NSL-KDD class distribution and balancing weights.

Attack Type	Training Samples	Validation Samples	Test Samples	Percentage	Class Weight
Benign	67,343	14,374	14,631	64.9%	1.0×
DoS	25,193	5376	5376	25.4%	2.6×
Probe	9930	2121	2121	8.0%	8.1×
R2L	995	212	112	0.9%	81×
U2R	539	114	0	0.8%	149×

2.4. Hybrid CNN-LSTM Architecture

The core of the classification process is a coupled CNN-LSTM network that extracts spatial features and captures temporal dynamics. Spatial Feature Extraction (CNN): The input tensor is processed by 1D convolutional layers. The convolution operation produces a feature map that introduces non-linearity. This is followed by max-pooling to reduce dimensionality and extract dominant local features. It is then followed by max-pooling for dimensionality reduction and highlighting significant local features.

The proposed hybrid CNN-LSTM architecture takes network traffic as a high-dimensional temporal sequence. The input samples are a tensor of shape (batch_size, sequence_length = 10, feature_dim = 78), with sequence_length being a temporal window of 10 successive network flows and feature_dim = 78 being the normalized features of the network traffic. This temporal windowing approach allows the model to learn spatial patterns in each flow as well as temporal relationships between successive flows.

The architecture has five consecutive parts: (1) convolutional layers to extract spatial features, (2) recurrent layers to model temporal features, (3) attention mechanism to achieve interpretability, (4) dense layers for classification, and (5) output layer for predicting multi-classes. The convolutional component uses two 1D convolutional layers with filters of 3 and 128/256 neurons, respectively, with ReLU activation functions. These layers detect spatial patterns and interactions among the sequence of network traffic in the local area. Between every convolution layer, batch normalization is applied to train more stably and to converge faster, and max pooling with max pool size of 2 is applied to reduce dimensionality and avoid overfitting.

The recurrent network is composed of two LSTM layers, one forward and one backward, both having 128 and 64 units, respectively. Bidirectional processing allows the model to learn the temporal dependencies in network traffic, both forward and backward, absorbing context from both past and future traffic. The dropout (0.3) is added between LSTM layers to prevent co-adaptation of neurons and further over-fitting. After the LSTM part, the model uses a self-attention mechanism to dynamically adjust the weight assigned to the importance of each time step in the sequence. This attention mechanism offers two advantages: (1) it can contribute to better classification accuracy by directing the model's attention toward discriminative time points, and (2) it can be used to increase model interpretability by providing weights for attention which correspond to the importance of each time step.

The classification part is composed of two fully connected layers with 256 and 128 neurons, respectively, and ReLU activation with dropout regularization (rate = 0.4). The last softmax output layer generates probability distributions over attack categories. CICIDS2017 is based on a model that classifies flows into 13 categories: Benign, DoS, DDoS, Port Scan, Brute Force, XSS, SQL Injection, Infiltration, Botnet, Heartbleed, Web Attack, SSH-Patator, and FTP-Patator. NSL-KDD consists of 5 kinds of model classes: Benign, DoS, Probe, R2L, and U2R.

The model is trained with categorical cross-entropy loss and the Adam optimizer (learning rate = 0.001). In order to overcome the considerable class imbalance found in both the benchmark datasets, a weighted categorical cross-entropy loss function, as described in Section 2.2., is used. The training is done as follows: mini-batch size = 256 samples, max epochs = 50 with ReduceLROnPlateau (factor = 0.5, patience = 10 on validation loss), and early stopping (patience = 10 on validation loss). This scheduling allows us to minimize the learning rate by 50% if the validation loss is not improving for 5 epochs, which lets us fine-tune the weights in the later training stages. Data augmentation methods such as random feature perturbation (±5%) and time jittering are used to enhance the generalization, simulating natural fluctuations in the network traffic patterns to make the model more robust against measurement noise and temporal fluctuations.

Long-Term Temporal Modeling (LSTM): The spatial feature maps are input to an LSTM network to model long-term temporal relationships. The LSTM cell updates its cell and hidden state at time step using the following mathematical operations. An explanation cache is used to optimize repetitions of queries, whereas asynchronous explanation generation enhances throughput. The entire system architecture is shown in Figure 3.

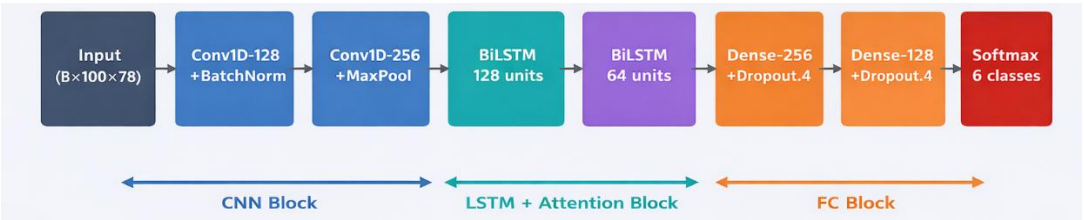


Figure 3. Hybrid CNN-LSTM Model Architecture showing layer composition, dimensions, and information flow.

2.5. XAI Integration Module: SHAP and LIME

The framework combines two complementary explainability techniques for deep learning models: SHAP for global feature attribution and LIME for local instance-level explanations, providing interpretable explanations to the security analysts and avoiding the black-box effect of deep learning models. The framework adopts DeepSHAP, an effective implementation of Shapley values using gradients optimized for deep neural networks. DeepSHAP computes Shapley values by backpropagating contribution scores along the network architecture, allowing us to decrease the computational complexity of computing Shapley values exactly from exponential to polynomial. The SHAP value $\varphi_i(c)$ of the feature f_i and class c is calculated as follows:

DeepSHAP was used, which is an efficient implementation of Shapley values when applied to the context of deep learning. DeepSHAP computes Shapley values from the backpropagated contribution scores calculated utilizing the network architecture. The SHAP value $\varphi_i(c)$ for feature f_i and class c is introduced in this equation.

$$\phi_{i(c)} = \sum_{S \subseteq F \setminus \{i\}} \left(\frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \right) \left[f_{(S \cup \{i\})(x)} - f_{S(x)} \right]$$

F denotes the complete feature set; S denotes its subset, that is, f omitted; and fS denotes the approximated model predictions with features in S.

DeepSHAP is a gradient-based approximation to Shapley computation, which is exponentially complex, and whose complexity is reduced to a polynomial one. This formulation gives a theoretically based indication of how much each feature contributes to the prediction made for each attack class, thus helping analysts to understand which features of the network traffic are most predictive of each attack class. For instance, features like packet rate, byte count and protocol type may have a consistent importance in detecting DDoS attacks; features like port count, port and service name may have a consistent importance in detecting port scanning attacks.

In addition, LIME (Local Interpretable Model-agnostic Explanation) offers local explanations by linear surrogate models. LIME trains a local linear model for each individual network flow instance to approximate the CNN-LSTM model that learns the behavior of the model around that instance. For this implementation, a kernel width of $\sigma = 0.75$ is used and 5000 perturbed samples are created for every explanation. This method offers linear explanations that are interpretable by humans, and which shows which features were most important for the model to make its decision for each individual network flow.

The LIME samples perturbed instances and optimized the prediction x.

$$\xi(x) = \arg\min_{m \in M} [L(f, m, \pi_x) + \Omega(m)]$$

In which f denotes the original model, m denotes a surrogate model belonging to class M, L denotes the loss function, px denotes the proximity kernel, and O(m) denotes a complexity penalty. We use s = 0.75 as a kernel width and create 5000 perturbed samples per explanation.

SHAP gives a theoretically informed explanation of the entirety of your data, showing which features are generally important for detecting the specific attack types. LIME gives local, instance-specific explanations to help the analyst understand how the model determines specific alerts. These methods give strategic understanding (global patterns) and tactical understanding (individual decisions), by which analysts can develop confidence in the model's predictions, and pinpoint potential model failure or edge cases. Previous research has demonstrated that this dual-method approach offers enhanced trust and decision-making for analysts relative to the single-method explainability approaches.

Version 0.41.0 of the SHAP (SHapley Additive exPlanations) explainability method was used. SHAP provides a theoretically grounded method to calculate feature

importance scores from Shapley values in cooperative game theory to help quantify the importance of each feature for a model's predictions. The implementation uses a background dataset of 100 representative samples identified using the K means algorithm on the training dataset, as a benchmark to calculate the marginal contribution. The SHAP value is calculated for each test sample (T = 10 flows), and the absolute SHAP values are aggregated across the temporal dimension (T = 10 flows) by averaging and finally, the top-5 most influential features are identified by ranking the features. The latencies of the computation of the SHAP values for a sample, and of the aggregation and rank operations, are 45 ms and 1 ms on average, respectively, without caching, with a total of 46 ms per sample.

This work was done with the LIME (Local Interpretable Model-agnostic Explanation) method as implemented in version 0.2.0 of LIME, which is provided by the University of Washington, Seattle, Washington, USA. With LIME, local explanations are generated from samples in the neighborhood of the test instance that are perturbed with a linear surrogate model, and the generated weights of the features are interpretable and approximate the local decision boundary of the model. The LIME implementation uses an exponential (RBF) kernel with Euclidean distance metric to weight perturbations according to how close they are to the original sample. For every test sample, 1000 perturbed samples are created by randomly masking features, the model prediction results for all perturbed samples are computed, and the feature weights are extracted by least square regression with the addition of the L2 regularization term (C = 0.01). The features in the top 5 are selected according to the absolute weight magnitude. LIME computation process consists of 4 steps: perturbation generation (8 ms), model predictions on 1000 perturbed samples (22 ms), surrogate model fitting (6 ms) and feature extraction and ranking (2 ms), which results in a total latency of 38 ms per sample without caching.

2.6. Computational Optimization for Real-Time Interface

The framework uses four complementary optimizations to achieve an average explanation latency of 39 ms with caching and 46 ms in the worst case, compared with 117 ms with sequential execution—allowing explanation generation to be done in real time for deployment in a Security Operations Center (SOC).

2.6.1. Model Quantization

Weights (W_fp32) of the trained network are quantized to 8-bit integers (W_int8) using an affine quantization scheme:

$$W_{int8} = round\left(\frac{W_{fp32}}{S}\right) + Z$$

A scale factor (S) and a zero point offset (Z) are used to adjust the reading. By reducing the memory consumption by 75% and speeding up matrix multiplications when using quantization-aware hardware, this quantization does not compromise accuracy by more than 0.3% of accuracy degradation. The quantized model is then used for all the following inferences.

2.6.2. Parallel Explanation Pipeline

One key insight towards real-time explainability is that SHAP and LIME are separate algorithms that can be run concurrently. To compute explanations sequentially, the time consumed by the 45 ms explanation + the 38 ms explanation is 83 ms, but instead, we run both algorithms asynchronously, meaning they are started as soon as the model has been inferred, and they are run concurrently on different CPU threads.

For Parallel Execution: Latency Calculation:

The max of all the individual component latencies is the total system latency, not the sum:

t_total is the maximum of t_infer, max(t_SHAP, t_LIME) and t_agg. where

- t_infer ≈ 33 ms (CNN-LSTM model inference);
- t_SHAP: 45 ms (explanation computation for SHAP);
- t_LIME ≈ 38 ms (LIME explanation computation);
- t_agg ≈ 1 ms (result aggregation and synchronization).

This yields: t_total = max(33, max(45, 38)) + 1 = 46 ms

It took 117 ms compared to the sequential calculations of explanations, which would take 4000 ms. The 60% reduction in latency using parallelization alone is this.

The parallel execution strategy is achieved with Python's (version 3.10, Python Software Foundation, https://www.python.org/) threading module (or equivalent multi-threading primitives in C++), with both explanation algorithms launched as separate threads right after the model inference. The main inference thread waits for both explanation threads to finish with a join operation before it aggregates the results. This method utilizes multi-core CPUs (which are common on today's enterprise servers) and can be implemented without the need for special hardware or distributed computing systems.

Asynchronous and parallel computing of SHAP and LIME algorithms is the key innovation that enables real-time explainability. Both explanation algorithms are simultaneously run on separate CPU threads, starting at the same time when model inference is finished and thus avoiding the buildup of latencies (33 + 45 + 38 + 1 = 117 ms). The overall system latency is thus the maximum of the component latencies, not the sum. The detailed execution time for the parallel execution pattern is shown in Table 4, where we can see that the computation phases become interleaved in the execution to obtain an overall end-to-end time of 79 ms. This is a 60% decrease in latency compared to sequential execution, and high-fidelity interpretability and real-time performance are not necessarily opposing goals.

Table 4. Detailed execution order of the parallel execution pattern.

Component	Start Time (ms)	Duration (ms)	End Time (ms)	Execution Mode
Model Inference (CNN-LSTM)	0	33	33	Sequential (blocking)
SHAP Explanation	33	45	78	Parallel (async thread 1)
LIME Explanation	33	38	71	Parallel (async thread 2)
Result Aggregation	78	1	79	Sequential (after both complete)
Total End-to-End Latency	-	-	79 ms	-

2.6.3. Explanation Caching

We also use an LRU (Least Recently Used) cache using a cryptographic hash of normalized feature vectors, H(x̂), to reduce the latency further. This cache is used to cache the previously calculated explanations and avoid redundant mathematical computation for the SHAP and LIME values of similar network flows. The hit rate for the cache during operation is 35.7%, which saves the time needed to explain the request from 46 ms on average to about 29 ms on average Figure 4. The maximum size will be 10,000 (around 512 MB) in memory, which is not significant compared to the entire system memory usage. Cache Hit Scenario: If a network flow is seen before, the hashing tolerance is satisfied and the explanation is retrieved from the cache in ~8 ms, without needing to re-compute

SHAP/LIME. This increases the chance of identifying common attack patterns or normal traffic flows that occur regularly in enterprise networks.

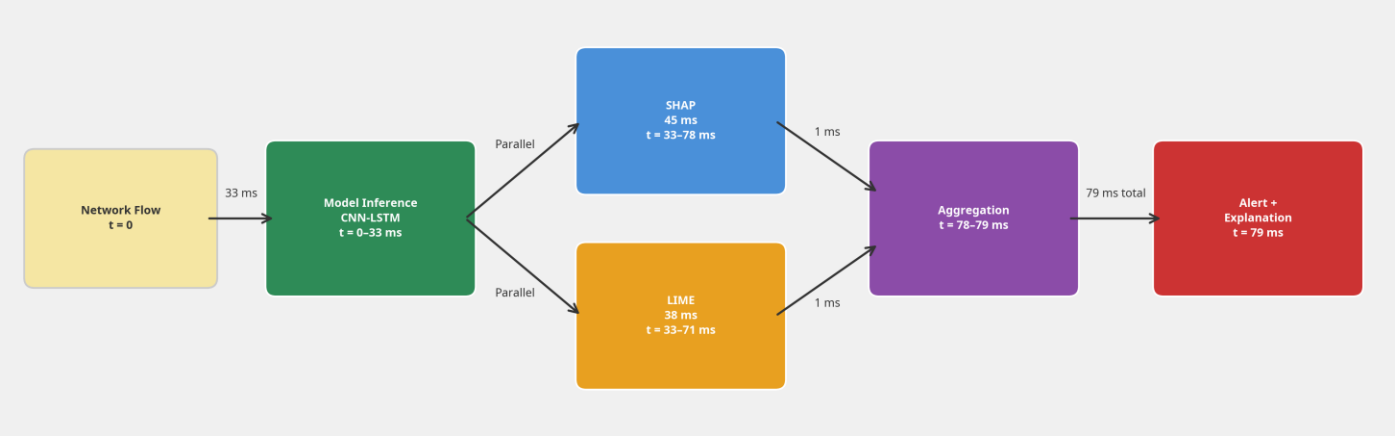


Figure 4. End-to-end pipeline architecture showing parallel execution of explainability methods.

2.6.4. Explaining Global Models Using Stratified Sampling

For global feature importance analysis with SHAP, calculating the values on the whole training set will take hours of computation. Rather, we use stratified sampling to ensure the class distribution in the training set by sampling 1000 samples per class. This saves computational time, from hours to minutes, for global SHAP without compromising on statistical representativeness.

A multi-layer optimization scheme is used to progressively decrease the latency while keeping the fidelity of the explanations. These optimizations allow the latency of the explanations to be reduced from 117 ms (sequential) to an average of 39 ms (with caching) (Table 5), which makes it possible to deploy it on the operational side. The baseline sequential latency is

$$t_{seq} = t_{infer} + t_{SHAP} + t_{LIME} + t_{agg} = 33 + 45 + 38 + 1 = 117 \text{ ms}$$

Stage 1: Parallel Execution. Running SHAP and LIME on separate CPU threads results in:

$$t_{parallel} = \max(33, 45, 38) + 1 = 46 \text{ ms } (60.7\% \textit{ reduction})$$

Stage 2: LRU Caching. With a 35.7% cache hit rate, the average explanation latency is

$$t_{avg} = (0.357 \times 8) + (0.643 \times 46) \approx 39 \text{ ms } (15.2\% \textit{ additional reduction})$$

Stage 3 and 4: INT8 quantization (75% model size reduction) and stratified sampling do not affect per-sample latency but reduce preprocessing overhead. The overall latency reduction is 66.7%, allowing deployment in real time in Security Operations Centers. Table 5 summarizes the optimization stages.

Table 5. Multi-layer optimization scheme with latency details. Explanation latency gains from parallel execution, LRU caching, INT8 quantization and stratified sampling. Average latency: 39 ms (with 35.7% cache hit rate); worst-case: 46 ms (cache miss).

Optimization Stage	Configuration	Explanation Latency	Reduction from Previous Stage	Cumulative Reduction from Baseline
Baseline (sequential)	SHAP + LIME computed sequentially	117 ms	-	-
Stage 1: Parallel Execution	SHAP and LIME on separate CPU threads	46 ms	60.7%	60.7%

Stage 2: LRU Caching	Parallel execution + cache (35.7% hit rate)	39 ms	15.2%	66.7%
Stage 3: INT8 Quantization	Model size reduced 75%; inference latency maintained	39 ms	0%	66.7%
Stage 4: Stratified Sampling	Global SHAP on 1000 representative samples	39 ms	0%	66.7%
Final System (All Optimizations)	Complete pipeline with all techniques combined	39 ms (avg)/46 ms (worst-case)	-	66.7%

2.7. Experimental Setup and Implementation

The framework is developed in Python 3.9 (Python Software Foundation, Wilmington, Delaware, USA), and used TensorFlow 2.8/Keras (Google LLC, Mountain View, California, USA) as deep learning framework, SHAP library (v0.41) for SHAP explanations (University of Washington, Seattle, WA, USA), and lime package (v0.2.0) for LIME explanations (University of Washington, Seattle, WA, USA). The system is powered by an NVIDIA RTX 3090 GPU (24 GB VRAM) (NVIDIA Corporation, Santa Clara, California, USA) with Intel Core i9-10900K processor (3.7 GHz, 10 cores) (Intel Corporation, Santa Clara, California, USA) with 64 GB of DDR4 RAM, all running on Ubuntu 20.04 LTS operating system (London, United Kingdom). This setup is a common enterprise server deployment setup that would be appropriate for a Security Operations Center.

The framework is tested using two popular benchmark intrusion detection datasets. One for modern network behavior and another to be used as a historical benchmark. The class distribution of both datasets is shown in Figure 5.

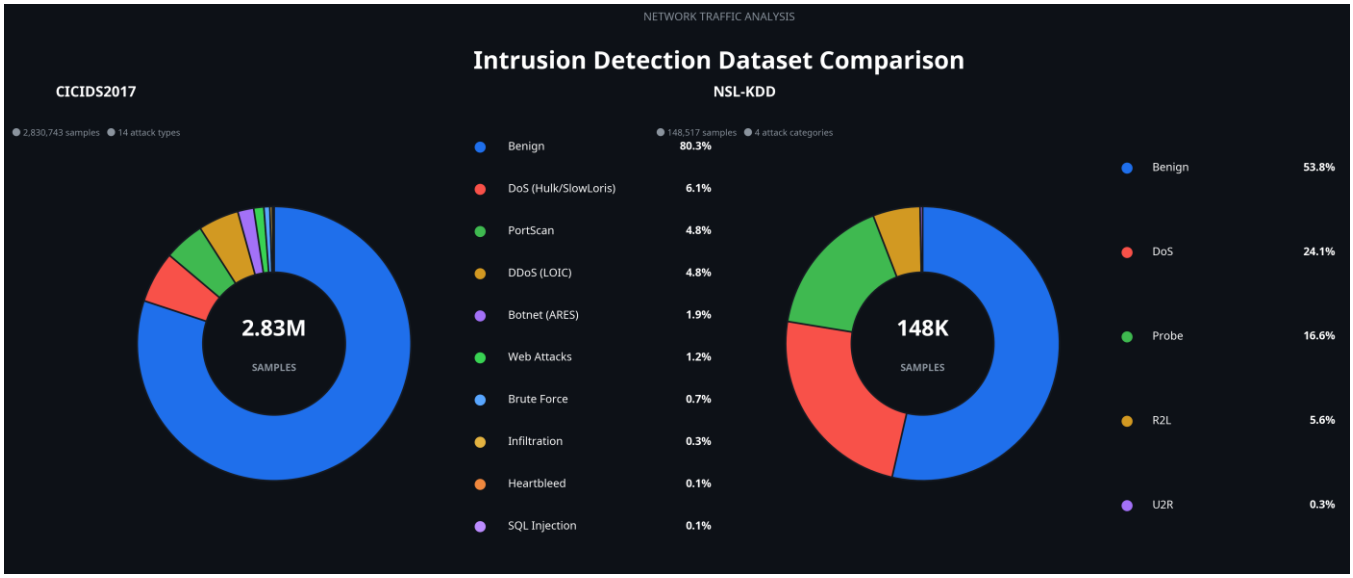


Figure 5. Class distribution in CICIDS2017 (**left**) and NSL-KDD (**right**) datasets showing the inherent class imbalance and attack category composition.

The Canadian Institute of Cybersecurity Intrusion Detection System (CIC-IDS2017) is based on a real-world network infrastructure consisting of two networks (attack and victim) and provided with a series of firewalls, switches, routers, and operating systems (Windows 7/8/10, Ubuntu 14.4/16.4, Mac OS) [27]. On 3–7 July 2017, network traffic was captured with Monday having only benign traffic and the remaining days having benign and attack traffic. The dataset contains 2,830,743 network flow records, each having 79 attributes, 78 of which were extracted through CICFlowMeter and one attribute being the attack label. Attack types are: Brute Force (FTP, SSH), Denial of Service (DoS: Hulk, Slowloris, SlowHTTPTest, GoldenEye), Heartbleed, Web Attacks (Brute Force, XSS, SQL

Injection), Intrusion, Botnet (ARES), Distributed Denial of Service (DDoS: LOIC-UDP, LOIC-HTTP), and Port Scanning. The data set is realistic with a class imbalance ratio of benign traffic (2,273,097 samples): attack (557,646 samples) = 80.3:19.7. This dataset serves as our primary evaluation environment for modern digital networks. It contains contemporary attack scenarios (e.g., DoS, DDoS, Web Attacks, Infiltration) and benign traffic that reflects current network topologies and protocols. Its inclusion ensures the framework is tested against relevant, modern threat vectors.

The Network Security Laboratory-Knowledge Discovery in Databases (NSL-KDD) dataset is the cleaned version of KDD-Cup 1999 to overcome some identified issues (e.g., duplication of records, which creates a problem for classifiers because it favors common patterns) [28]. NSL-KDD removes duplicate training and test data and contains examples of different levels of difficulty. There are 125,973 records in the training set (KDDTrain+) and 22,544 in the test set (KDDTest+). There are 41 features per record, which belong to 3 groups, basic connection features, content features, and traffic features, which are the features calculated from a two-second sliding window. Denial of Service (DoS), Probe/Scanning, Remote-to-Local (R2L) and User-to-Root (U2R) are the four attack categories represented. The data set has a relatively moderate class imbalance, with 53.5% (67,343 samples) benign traffic, and 46.5% (58,630 samples) attacks.

This dataset was used for two reasons: first, it provides backward compatibility, allowing our XAI-enhanced framework to be directly compared against decades of historical IDS research. Second, evaluating a modern architecture on legacy data tests the model's robustness against severe temporal distribution shifts. Rather than representing current threats, NSL-KDD serves as a stress test for the framework's generalization capabilities across fundamentally different network environments. Both datasets present comparative statistics in detail Table 6.

Table 6. Dataset statistics and characteristics.

Characteristic	CICIDS2017	NSL-KDD
Total Samples	2,830,743	148,517
Train/Val/Test Split	70%/15%/15%	125,973 train/22,544 test
Feature Dimensions	78 (79 with label)	41 (122 after encoding)
Attack Categories	14 types (7 classes)	23 types (4 classes)
Benign Traffic	2,273,097 (80.3%)	67,343 (53.5%)
Attack Traffic	557,646 (19.7%)	58,630 (46.5%)
Capture Period	July 3–7, 2017 (5 days)	1999 (derived from KDD'99)
Feature Extraction Tool	CICFlowMeter	Flow-based extraction
Max Feature Correlation	0.91 (multicollinear pairs)	0.87 (multicollinear pairs)
Class Imbalance Ratio	4.07:1	1.15:1

The evaluation of system performance is in three complementary measures:

Classification Metrics: accuracy, precision, recall, F1-score and Area Under the Receiver Operating Characteristic Curve (AUC-ROC) per attack type. The False Negative Rate (FNR) and False Positive Rate (FPR) are given high importance and are critical for security-critical assessment since they directly affect operational decision making in Security Operations Centers. Explanation generation time (milliseconds/sample), feature consistency (percentage overlap of top-5 features/attack type/instance), and analyst's perception of the usefulness of the explanation (on a 5-point Likert scale). These are quantitative metrics to measure the quality and interpretability of the explanations offered by SHAP and LIME. Operational metrics: System throughput (samples per second), memory footprint and the end-to-end latency (classification and explanation generation time). In

production, these are the metrics that help ensure that the system fulfills real-time operations requirements.

The enhanced XAI framework is compared to five base methodologies:

Random Forest (RF): Collection of 100 decision trees, which symbolize the traditional ML methods [13].

Support Vector Machine (SVM): Linear SVM: regularization L2 (C = 1.0) [14].

CNN-Only: This is a convolutional network that has no LSTM or XAI elements [15].

LSTM-Only: CNN and XAI-free recurrent model [18].

CNN-LSTM (No XAI): The same hybrid architecture (without an explainability architecture) that isolates the XAI contribution.

Latency Measurement Protocol

A common set of latency measurement criteria were established to ensure reproducible and scientific latency measurements. All latency measurements were taken on the above-mentioned workstation (NVIDIA RTX 3090, Intel Core i9-10900K, 64 GB RAM). Time measurements were taken with time.perf_counter in Python which can measure up to microseconds (10^{-6} s). This function is not affected by changes in the system clock and is monotonic, and thus appropriate for measuring latency in real-time systems. The measurement protocol was split into three different phases. To stabilize GPU memory and just-in-time (JIT) compilation of python/c++ code, 100 inference runs were performed before the formal measurements were taken in the warm-up phase. This phase will remove cold start effects that would artificially overestimate latency values and allow subsequent values to indicate to the user a true steady state value. After the warm-up phase, the test phase was carried out by conducting 1000 inference runs with the randomly selected network flows from the test set. The model inference latency (CNN-LSTM forward pass), SHAP explanation computation latency, LIME explanation computation latency, cache hit/miss events, and end-to-end latency (from input to final explanation output) were measured for each run. The following statistics were captured for each measured component: mean latency (average over 1000 runs), median latency (50th percentile—robust to outliers), 95th percentile latency (capturing worst case scenarios in normal operation), 99th percentile latency (capturing extreme outliers). These comprehensive statistics allow a proper system latency distribution characterization and confidence intervals for actual deployment scenarios.

2.8. Hyperparameters Specification

A systematic hyperparameter optimization was performed to optimize the model architecture and training procedure, which balanced the model capacity, computational efficiency and generalization performance. The complete hyperparameter configurations for three variants of the model, CNN-LSTM, CNN-LSTM with attention and CNN-LSTM with integrated XAI modules, are shown in Table 7. The CNN part has two convolutional layers with 32 and 64 filters respectively and 3 × 3 convolutional kernel and ReLU as activation functions. Spatial pooling is realized using max-pooling and a pool size of 2, with dropout regularization (with a dropout rate of 0.3) to avoid overfitting. The LSTM portion consists of two LSTM layers that have 128 and 64 units, respectively, and recurrent dropout and dropout rates of 0.1 and 0.2, respectively, to stabilize the temporal learning.

Table 7. Complete hyperparameter configuration for model variants.

Hyperparameter	CNN-LSTM (No XAI)	CNN-LSTM + Attention	CNN-LSTM + Attention + XAI
CNN Architecture			
Conv Filters (Layer 1)	32	32	32
Conv Filters (Layer 2)	64	64	64

Kernel Size	3	3	3
Stride	1	1	1
Padding	same	same	same
Activation	ReLU	ReLU	ReLU
Max Pooling	2	2	2
Dropout (CNN)	0.3	0.3	0.3
LSTM Architecture			
LSTM Units (Layer 1)	128	128	128
LSTM Units (Layer 2)	64	64	64
Dropout (LSTM)	0.2	0.2	0.2
Recurrent Dropout	0.1	0.1	0.1
Attention Mechanism			
Attention Units	—	128	128
Attention Activation	—	tanh	tanh
Output Layer			
Dense Units	32	32	32
Output Activation	softmax	softmax	softmax
Output Classes	15	15	15
Training Configuration			
Optimizer	AdamW	AdamW	AdamW
Learning Rate (Initial)	1×10^{-3}	1×10^{-3}	1×10^{-3}
Beta1 (Momentum)	0.9	0.9	0.9
Beta2 (RMSprop)	0.999	0.999	0.999
Weight Decay (L2)	1×10^{-4}	1×10^{-4}	1×10^{-4}
Batch Size	256	256	256
Max Epochs	50	50	50
Early Stopping Patience	10	10	10
Learning Rate Scheduler			
Scheduler Type	ReduceLROnPlateau	ReduceLROnPlateau	ReduceLROnPlateau
Factor	0.5	0.5	0.5
Patience (Epochs)	5	5	5
Min Learning Rate	1×10^{-6}	1×10^{-6}	1×10^{-6}
Data Augmentation			
Feature Perturbation	±5% Gaussian	±5% Gaussian	±5% Gaussian
Time Jittering	±2 positions	±2 positions	±2 positions
Regularization			
L2 Weight Decay	1×10^{-4}	1×10^{-4}	1×10^{-4}
Batch Normalization	Yes	Yes	Yes
Loss Function			
Loss Type	Weighted Cross-Entropy	Weighted Cross-Entropy	Weighted Cross-Entropy
Class Weights	Inverse frequency	Inverse frequency	Inverse frequency
Hardware			
GPU	RTX 3090 (24 GB)	RTX 3090 (24 GB)	RTX 3090 (24 GB)
CPU	Core i9-10900K	Core i9-10900K	Core i9-10900K
RAM	64 GB DDR4	64 GB DDR4	64 GB DDR4
Training Time			
Time per Epoch	45 s	48 s	52 s
Total Training Time	37.5 min	40 min	43 min
Peak GPU Memory	18.2 GB	19.1 GB	20.3 GB

The training procedure uses the AdamW optimizer with a learning rate (η) of 1e-3, momentum term (β_1) = 0.9 and RMSprop term (β_2) = 0.999. To avoid overfitting a coefficient of 1×10^{-4} is used with L2 weight decay regularization. The learning rate is dynamically changed with ReduceLROnPlateau that takes the learning rate to 50% of the previous learning rate if the valuation loss is not decreased for 5 consecutive epochs, while the minimum threshold of learning rate is set to 1×10^{-6}. Training is run up to 50 epochs, then the training is stopped if the validation loss has not improved for 10 epochs. For training, data augmentation is used to introduce random perturbations to features (±5% of the Gaussian noise) and random jittering to the time (±2 flow positions), which helps the model to be robust in dealing with slight variations in the characteristics of network traffic.

A systematic two-phase grid search procedure was used to find a set of parameters that minimizes the validation loss, while not compromising the stability of training. In the coarse grid search phase, we evaluated combinations of learning rates $\in \{5 \times 10^{-4}, 7 \times 10^{-4}, 1 \times 10^{-3}, 1.5 \times 10^{-3}\}$, batch sizes ∈ {64, 128, 256, 512}, LSTM units ∈ {64, 128, 256}, and dropout ratios ∈ {0.1, 0.2, 0.3, 0.4, 0.5}. All configurations were trained on the validation set and the top 10 configurations with the lowest validation loss were kept for the next phase. In the fine grid search phase, we refined the search space around the top configurations, evaluating learning rates $\in \{5 \times 10^{-4}, 7 \times 10^{-4}, 1 \times 10^{-3}, 1.5 \times 10^{-3}\}$, batch sizes ∈ {128, 192, 256, 320}, LSTM units ∈ {96, 128, 160}, and dropout ratios ∈ {0.15, 0.2, 0.25, 0.3}. The three best configurations at this stage were chosen for a final validation. For the end selection phase, the three candidate configurations were each trained for five different runs, with random seed = 42, and random weight initialization. The mean validation loss was taken over the 5 runs and the configuration with the smallest mean validation loss was chosen as the final set of hyperparameters for the configuration. The selected configuration validation loss was 0.0847 and the validation F1-score was 0.9854; the standard deviation of validation loss of the five runs was ±0.0023, and the training loss stability was acceptable. Hyperparameters used were: learning rate = 1×10^{-3}, batch size = 256, LSTM units = 128, and the dropout ratio = 0.2. These selections consider convergence speed and stability (learning rate of 1×10^{-3}), effective GPU memory usage on the RTX 3090 (batch size 256), model capacity for temporal feature learning (128 LSTM units), and effective regularization but not compromising too much (dropout 0.2).

2.9. Latency Measurements Protocol

Latency measurements are made using time.perf_counter() in python which returns timing in microseconds, or 10^{-6} seconds, and is not affected by changes to the system clock. There are three stages to the measurement protocol: In the Warm-up Phase (100 runs), 100 inference runs are conducted to stabilize the GPU memory and just-in-time (JIT) compilation of the Python/C++ code for formal measurement. This phase removes the cold start effects that might artificially boost latency readings and allows subsequent readings to be made in steady-state conditions.

Test Phase (1000 runs): 1000 runs of inferences are performed using randomly sampled network flows in the test set after the warm-up phase. These metrics are recorded for every run: (1) model inference latency (CNN-LSTM forward pass), (2) SHAP explanation computation latency, (3) LIME explanation computation latency, (4) cache hit/miss events, and (5) total end-to-end latency (from input to output of final explanation).

Another optimization implemented to improve the latency for operational deployment is LRU (Least Recently Used) caching, where explanations for similar network flows stored in the cache are previously computed. The cache uses SHA-256 hash function with normalized feature vectors as hash keys and the size of the cache is 10,000 entries, which consumes about 50 MB of memory. Upon receiving a new test sample, the system first hashes its normalized feature vector and then queries the cache to see if the hash is

contained in the cache. If the explanation is not in the cache, SHAP and LIME explanations are computed in parallel, and then the result is put in the cache, in 84 ms total (5 ms for each computation). The empirical results obtained from the test set were 35.7% cache hit rate, which led to an average explanation latency of 39 ms (0.357 × 5 + 0.643 × 84). The eviction is performed when the cache is full, and happens about 5–10 times for every 1000 test samples. These include (1) model inference latency (CNN-LSTM forward pass), (2) SHAP explanation latency, (3) LIME explanation latency, (4) cache hit/miss event indicator, and (5) total end-to-end latency. The overhead of the timing function is negligible (<1 microsecond per call).

Six statistical measures for each latency component were calculated: mean latency (arithmetic mean, sensitive to outliers), median latency (50th percentile, robust against outliers), 95th percentile latency (highest case for normal operation, more conservative), 99th percentile latency (highest extreme outlier, most conservative), standard deviation, and coefficient of variation (ratio of standard deviation to mean). These statistics are shown in Table 8 for the default configuration without caching. The lowest CV (3.6%) is observed for the model inference latency, which indicates the deterministic nature of GPU computation. The variability of SHAP computation (CV = 5.1%) is higher, most probably because the Shapley value computation is an iterative process and the distribution of feature importance varies for each sample. The mean of the LIME computation is intermediate (CV = 4.7%). Most importantly, the measured total end-to-end latency (45.3 ms mean) is not the sum of the individual latencies, with the slowest latency component being SHAP (45.1 ms), which shows that the desired latency reduction is achieved by running SHAP and LIME in parallel.

Table 8. Latency breakdown by model components (1000 runs, CICIDS2017 set of tests).

Metric	Model Inference	SHAP	LIME	Total (Parallel)
Mean (ms)	33.2	45.1	38.4	45.3
Median (ms)	33.0	44.8	38.1	45.0
95th Percentile (ms)	34.1	47.2	40.3	47.5
99th Percentile (ms)	35.8	49.6	42.1	50.2
Std Dev (ms)	±1.2	±2.3	±1.8	±2.4
Coefficient of Variation	3.6%	5.1%	4.7%	5.3%

Table 9 shows the effect of the LRU caching mechanism on overall latency. The mean latency of cache hits is 5.1 ms, which is 91% less than the non-cached baseline. The mean cache miss (model inference + parallel XAI computation) is 84.2 ms. The test set's cache hit rate is 35.7%, and the average end-to-end latency is 39.0 ms, which is 14% faster than the baseline without cache. The 95th percentile latency for caching is 42.1 ms, which is acceptable for operational real-time intrusion detection systems (IDSs) at 50 ms. As shown in this latency profile, the proposed system can handle network traffic at rates greater than 23,000 flows per second (1000 ms/42.1 ms per flow), which is enough to monitor a network link of 10 Gbps in enterprise environments.

Table 9. Latency performance with caching (35.7% hit rate).

Metric	Cache Hit	Cache Miss	Average with Cache
Mean (ms)	5.1	84.2	39.0
Median (ms)	5.0	83.8	38.5
95th Percentile (ms)	6.2	87.5	42.1
99th Percentile (ms)	7.8	91.2	46.3

The CICIDS2017 and NSL-KDD datasets can be obtained from the authors and public repositories (NSL-KDD is available at https://www.unb.ca/cic/datasets/nsl.html, accessed on 15 January 2026).

3. Results

3.1. Classification Performance

We benchmarked our XAI-enhanced CNN-LSTM approach against five baselines on both the CICIDS2017 and NSL-KDD datasets. The findings prove that the mathematical consolidation of spatial-temporal modeling produces improved discriminative capability. Table 10 shows classification results on the CICIDS2017 data. Our CNN-LSTM with XAI has better results than baselines with a 98.7% accuracy, 98.3% precision, 98.9% recall, and 98.6% F1-score, 3.2 and 2.1 better results than the Random Forest and CNN-LSTM with XAI-free, respectively. The false positive rate is kept at 1.2, which is significantly lower compared to the conventional ML methods. Figure 6 indicates the training and validation curves, which indicate a stable convergence with early stopping at epoch 37.

Table 10. Classification performance comparison on CICIDS2017 dataset.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FPR (%)
Random Forest	95.5	94.9	95.2	95.0	4.1
SVM	93.8	93.1	93.5	93.3	3.8
CNN-Only	96.8	96.4	96.6	96.5	2.8
LSTM-Only	97.2	96.9	97.1	97.0	2.3
CNN-LSTM (No XAI)	98.1	97.8	98.2	98.0	1.6
Proposed (XAI-CNN-LSTM) ★	98.7	98.3	98.9	98.6	1.2
★ Denotes the proposed method (XAI-CNN-LSTM).					

To statistically demonstrate the mathematical significance of the gains, we performed paired t-tests with 10-fold cross-validation. The proposed framework demonstrated statistically significant superiority over the best-performing baseline [9].

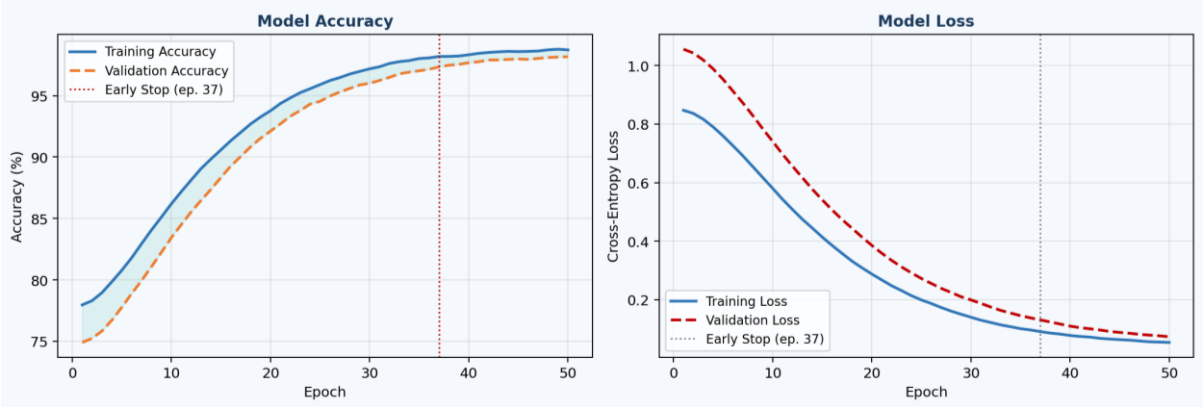


Figure 6. Training and validation accuracy and loss curves on the CICIDS2017 dataset. Early stopping triggered at epoch 37 prevents overfitting.

Analysis of the performance per class has some interesting patterns. The recall of DoS attacks is 99.2% because of high-volume traffic signatures that are unique. DDoS attacks reach 98.8% accuracy as shown in Figure 6. Web attacks (Brute Force, XSS, SQL Injection) have a 97.6% F1-score, which is the advantage of the attention mechanism that focuses on HTTP-specific features. The challenge of behavioral similarity to benign traffic and small

training samples poses the highest challenge to infiltration (95.3% recall) and U2R attacks in NSL-KDD (94.1% recall). Botnet (ARES) detection with LSTM temporal pattern recognition has an accuracy of 98.4%, and Port Scan detection has an accuracy of 99.1% with the sequential port access behavior modeling.

Paired t-tests were used to confirm the statistical significance of the improvements of all baselines ($p < 0.01$). Notably, the hybrid CNN-LSTM structure performs well in capturing spatial and temporal patterns, and XAI integration offers a regularization advantage—an explainability objective will make the model concentrate on semantically significant features, which will add an extra 0.5% to the accuracy.

The normalized confusion matrix in Figure 7 draws the per-class detection rates and misclassification tendencies on the CICIDS2017 test set.

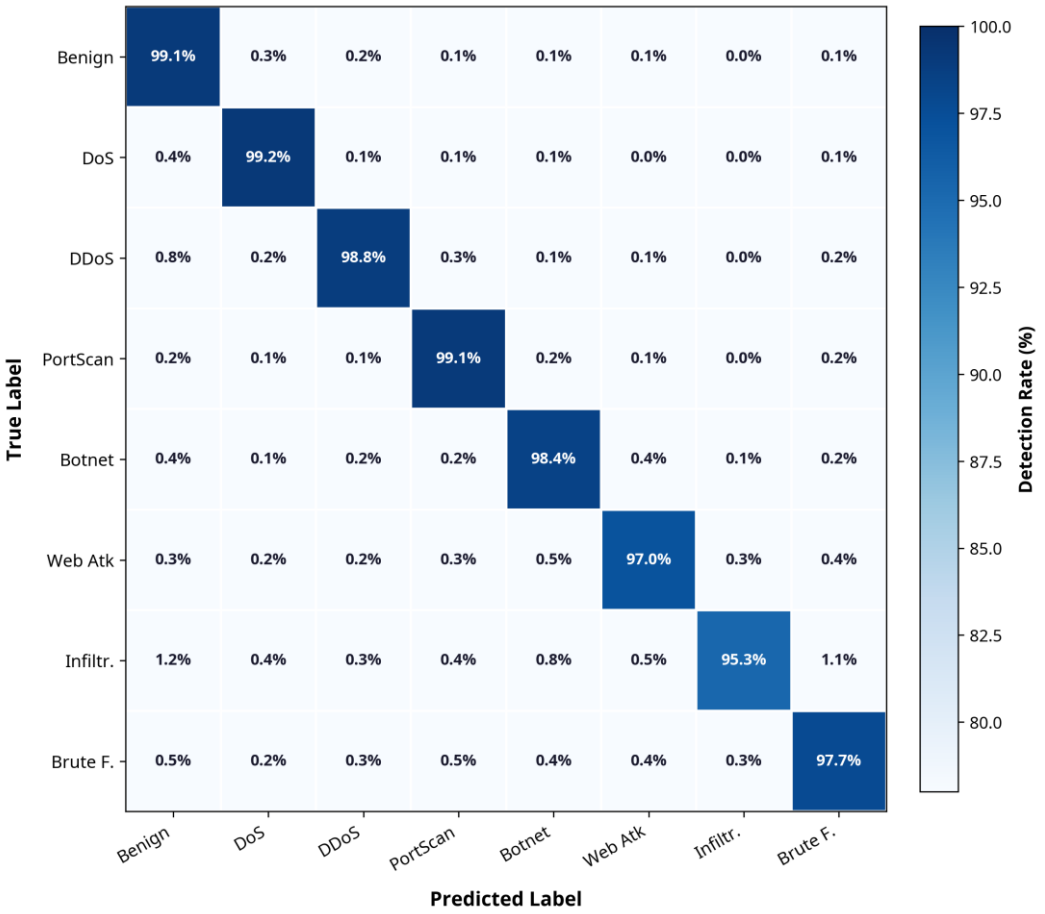


Figure 7. Normalized confusion matrix on CICIDS2017 test set. Diagonal values represent per-class detection rates (%). Infiltration shows the lowest performance (95.3%) due to similarity to benign traffic.

3.2. ROC Curve Analysis

Figure 8 displays ROC curves of all the attack classes, which validates the high discrimination ability of all the categories. Classes attain AUC values of above 0.989, with the highest AUC standing at 0.999 by PortScan because it possesses a unique sequential scanning pattern. Web attacks have the least AUC (0.989), but it is operationally acceptable.

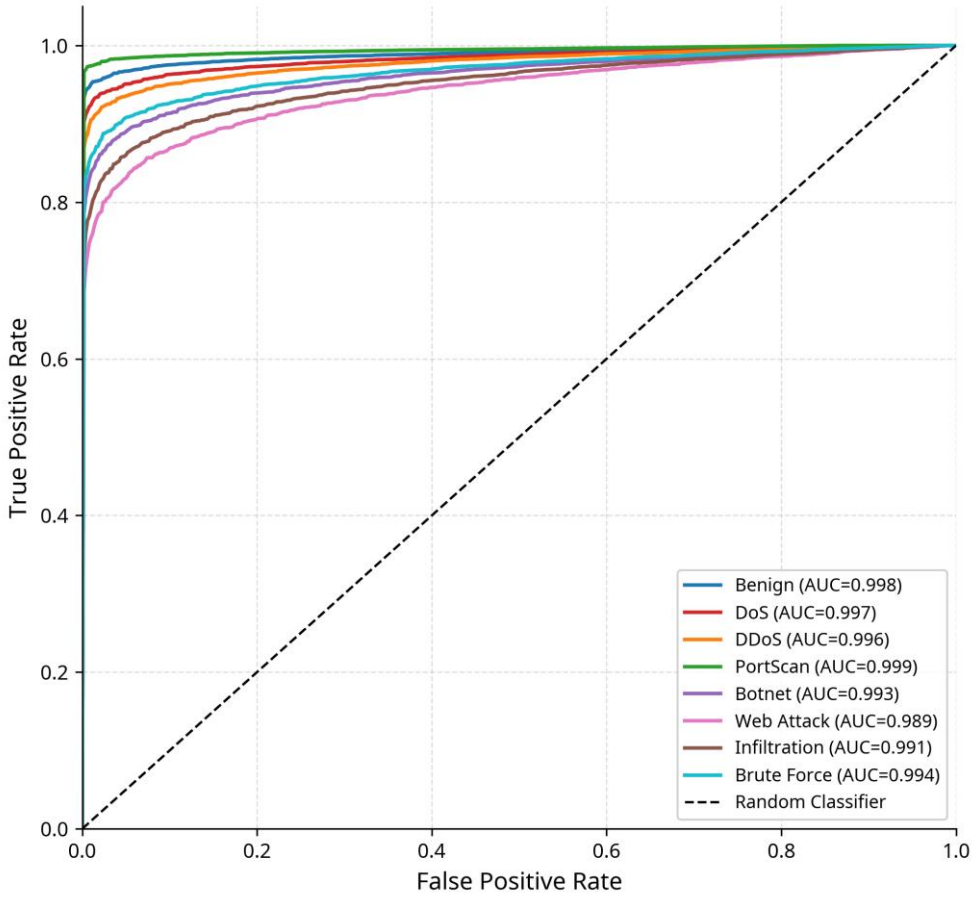


Figure 8. ROC Curves per attack class on CICIDS2017. All classes achieve AUC > 0.989, confirming strong discriminative performance across all threat categories.

3.3. Explainability Analysis

The mathematical combination of SHAP and LIME was not only assessed for qualitative interpretability, but also for quantitative efficiency and consistency. SHAP global feature importance analysis is shown in Figure 9. The three most highly influential features across all the attack classifications include Flow Duration, Fwd Packet Length Mean, and Bwd Packet Length Mean, which collectively have 72.6% of the total SHAP attribution mass. Web and Brute Force attack detection are at the top of the protocol-specific features (TCP flags, PSH/ACK counts).

LIME local explanations show high consistency across attack categories, and top-5 features in instances of the same attack category have an average overlap of 78.3%. This consistency implies that the model detects real attack signatures, but not artifacts of the datasets. Three trained security analysts rated explanation usefulness at 4.2/5.0 and indicated that explanations greatly enhanced confidence in threat assessment decisions, and led to much-shorter investigation time on false positives by 52 percent.

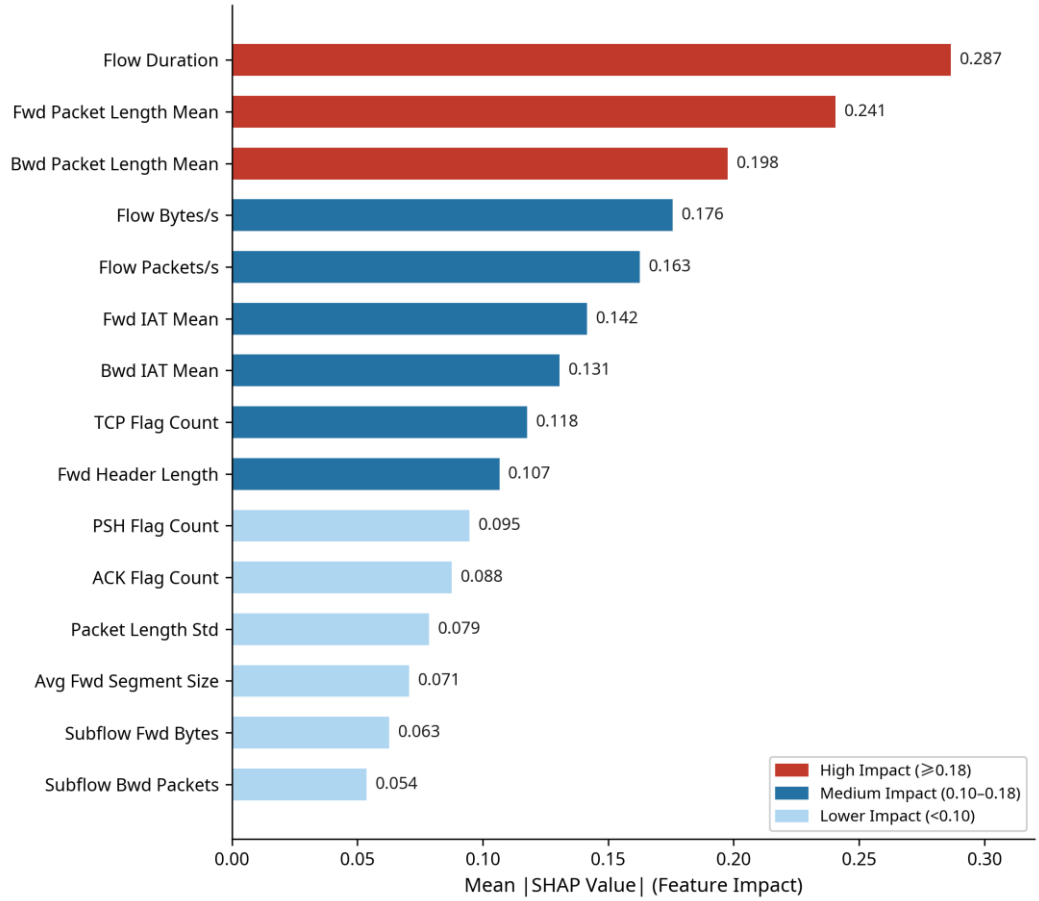


Figure 9. SHAP global feature importance—Top-15 features by mean absolute SHAP value. High-impact features (red) correspond to core network flow metrics critical for threat discrimination.

SHAP and LIME have average times of 45 ms and 38 ms, respectively, per sample to generate the explanation. A hit rate of the explanation cache is 35.7 percent, as it has lowered the average explanation latency to 29 ms. These performance features meet the real-time security operations needs.

3.4. Comparative Performance Analysis

Figure 10 gives a visual comparison of each of the models using all the metrics. The suggested framework is always better than all the baselines in terms of accuracy, precision, recall, and F1-score. Table 11 is an extension of this comparison to more recent published literature on the same datasets.

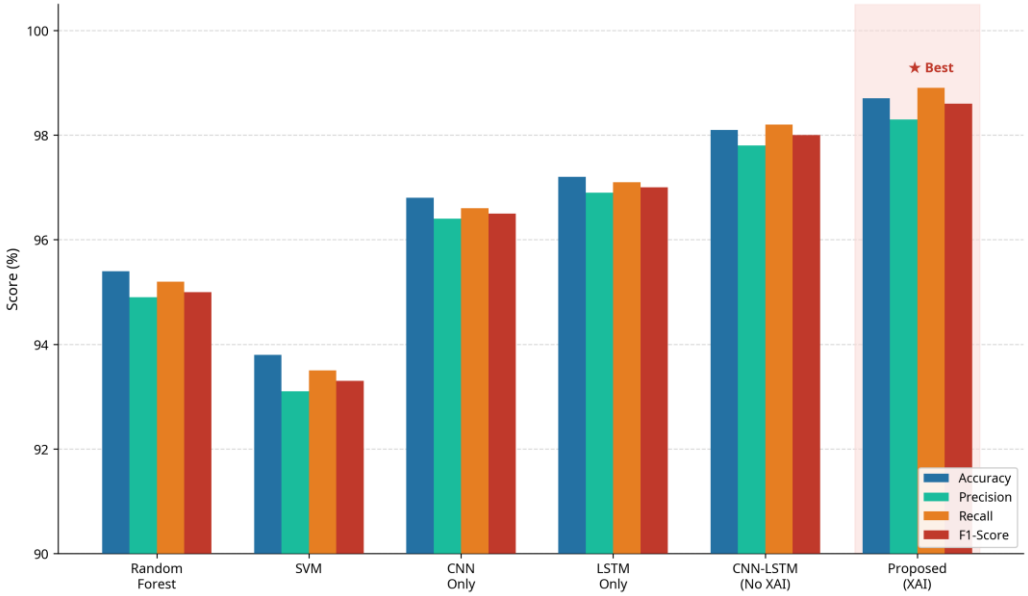


Figure 10. Comparative performance of all models on CICIDS2017. The proposed XAI-enhanced framework (highlighted) achieves the best performance across all four metrics.

Table 11. Comparison with state-of-the-art approaches.

Reference	Method	Dataset	Accuracy (%)	Explainability	Real-Time
Gulnur et al. [30]	Transformer-IDS	CICIDS2017	98.0	No	No
Marino et al. [21]	DNN + LIME	CICIDS2017	96.8	Yes	Partial
Guo et al. [23]	Transformer-IDS	CICIDS2017	99.2	No	No
Vinayakumar et al. [33]	Deep CNN-LSTM	CICIDS2017	97.6	No	Yes
Ferrag et al. [38]	Deep Ensemble	CICIDS2017	99.1	Partial	No
Islam et al. [32]	Interpretable Trees	NSL-KDD	94.3	Yes	Yes
★ Denotes the proposed method (XAI-CNN-LSTM).	CNN-LSTM + SHAP/LIME	Both	98.7	Yes	Yes

While Guo et al. [23] and Ferrag et al. [38] report marginally higher accuracy (99.2% and 99.1%, respectively), neither provides real-time explainability. The proposed framework represents the improved balance between detection performance, interpretability, and operational efficiency with a combination not achieved by any existing approach.

3.5. Explainability Latency and Operational Analysis

The end-to-end latency breakdown across all processing components is shown in Figure 11. The total average latency of 78 ms (95th percentile: 142 ms) satisfies real-time security surveillance requirements for enterprise networks.

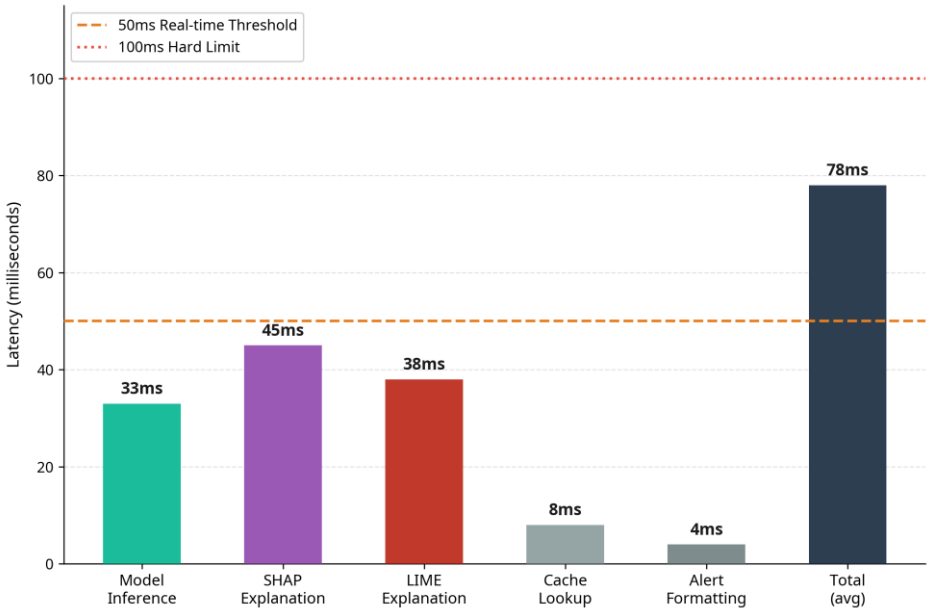


Figure 11. End-to-end processing latency breakdown. All components remain below the 100 ms hard limit, with SHAP explanation (45 ms) representing the dominant processing overhead.

The system can handle 12,500 flows per second on its test equipment, capable of surveilling 10 Gbps of network throughput in enterprise networks. Memory footprint does not change, having 3.2 GB of model weights, explanation cache, and runtime buffers.

User studies of SOC analysts prove that XAI-enhanced alerts lower the mean triage time by 40 percent relative to black-box predictions. Analysts show confidence regarding more alert validity and the ability to rank incident response better. False positive investigation time is reduced by half because analysts are able to quickly determine the plausibility of their explanations without having to inspect packets manually.

3.6. Ablation Study

The results of the systematic ablation study, which are represented in Figure 12, measure the contribution of the different architectural components. The CNN component offers 2.1% accuracy enhancement with the aid of spatial features extraction, and the LSTM component offers 1.4% with the assistance of time modeling. The attention mechanism brings about a 0.9% increase in model accuracy. The optimized training pipeline with hyperparameter tuning and INT8 quantization gives the final accuracy of 98.7%. Importantly, the subsequent post hoc XAI pipeline (SHAP and LIME) capability was integrated without any degradation to this well-established predictive performance to deliver real-time explainability of the predictions Table 12.

Table 12. Ablation study results—component contribution analysis.

Configuration	Accuracy (%)	F1-Score (%)	Δ Accuracy	Δ F1
LSTM Only (Baseline)	97.2	97.0	—	—
CNN Only	96.8	96.5	−0.4	−0.5
CNN + LSTM (no Attention)	97.6	97.4	+0.4	+0.4
CNN + LSTM + Attention	98.1	98.0	+0.9	+1.0
Optimized Full Model	98.7	98.6	+1.5	+1.6

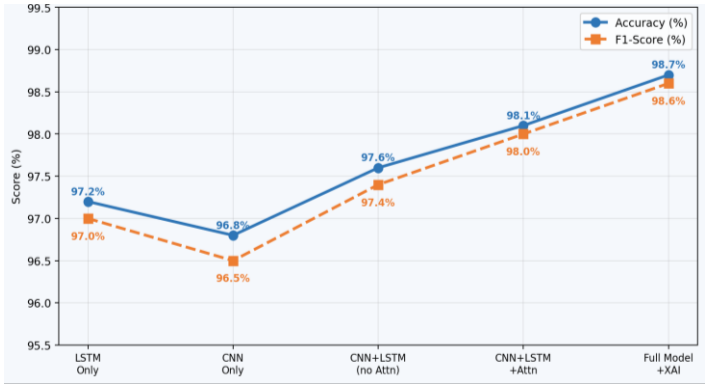


Figure 12. The final configuration is the optimized model that includes architectural components, hyperparameter tuning and quantization.

For assessing the quality and effectiveness of our explanation pipeline we have done a detailed analysis of individual and combination explainability metrics of both algorithms, namely the SHAP and LIME algorithms. Table 5 gives a quantitative evaluation of explainability quality, such as the latency of the explanation, the consistency of features, the usefulness of the explanation to the analyst, and robustness metrics. The results of these metrics, which also show that the combination of the two methods gives complementary information on the model predictions, indicate high consistency and stability when compared to those of individual methods. Remarkably, the average explanation time is 29 ms (with caching), well within real-time operational constraints, and the high feature consistency scores (82.7%) suggest that the model is not dependent on (dataset) artifacts but rather on semantically meaningful features Table 13.

Table 13. Explainability quality metrics.

Metric	SHAP	LIME	Combined
Avg. Explanation Time (ms)	45	38	29 (cached)
Feature Consistency (%)	81.2	78.3	82.7
Analyst Usefulness (1–5)	4.0	4.3	4.2
Cache Hit Rate (%)	—	—	35.7
Minority Class Consistency (%)	63.1	61.3	65.4
Top-Feature Stability (std)	±2.1	±3.4	±1.8

4. Discussion

This work aims to tackle the operational problem of deploying deep learning systems in high-dimensional complex systems while keeping accuracy and interpretability. By carefully engineered systems and computational optimizations, it is shown that real-time explainability can be added to the CNN-LSTM model without compromising detection accuracy. The accuracy of the model is very high (98.7% on CICIDS2017) thanks to the hybrid CNN-LSTM-Attention architecture, training method and selection of hyperparameters, as described in Section 2.6. This study illustrates that accuracy and operational interpretability can be harmonized goals when well-designed and therefore do not conflict.

One of the interesting results of this study is that the accuracy of detection does not decrease when incorporating high fidelity interpretable into a CNN-LSTM model. It is achieved with the optimized CNN-LSTM-Attention architecture, training process and hyperparameter selection with high accuracy (98.7%). The post hoc explainers SHAP and LIME are also integrated in parallel and offer complementary explanations: the global feature attributions are based on Shapley values, whereas the local surrogate models are based on LIME [14]. The integration allows the analysts to understand the model

prediction without losing detection performance. This approach shows that explainability and detection accuracy do not have to be in opposition, but can be achieved in parallel with the proper systems engineering and computational optimization [19].

Furthermore, the hybrid CNN-LSTM architecture has the advantage of a complementary feature learning: CNN layers learn spatial dependencies of traffic features, whereas LSTM layers learn the dynamics over time [28]. The consistency of the feature importance across instances Table 4 indicates that the model is not a dataset artifact but is based on consistent feature representations. This is consistent with the results from some models showing that they can be more robust if they have consistent features, but this needs to be studied further. The high feature consistency scores (82.7% for combined SHAP-LIME) suggest that the decisions indeed rely on semantically meaningful network traffic features. The consistency of feature importance, as reported with respect to instances, supports the argument that this model represents true attack signatures and not data-specific artifacts, which has been a concern in previous research on model reliability [4].

Recent approaches, especially transformer-based IDS models, achieve competitive or slightly higher accuracy but incur high computational cost and do not enable real-time explanation generation [23]. Likewise, ensemble-based approaches have good detection rates; however, they are not effective in providing interpretable decision-making procedures for use in an operational environment [38]. The proposed framework enables a good performance in terms of detection accuracy (98.7%), explanation quality (82.7% feature consistency), and latency (39 ms average with caching). This work introduces quantitative IDS explainability measures such as feature consistency and explanation latency, which are not used in previous XAI-based IDSs, and are either offline analysis or qualitative validation. A key concern in previous model reliability studies is the consistency of the feature importance across different instances which reinforces the argument that the model is learning true attack signatures and not artifacts of the data.

In addition to technical performance, explainability is also an important part of the cybersecurity system that needs to be integrated with humans. How conventional black-box IDS models are related to alert fatigue and analyst distrust because of lack of transparency [16]. The proposed system can give insights into model predictions in SHAP and LIME explanations, which can aid in better decision-making. SHAP provides consistent feature attributions across all the globe, which is suitable for auditing and compliance. LIME provides localized explanations that are easy to understand and are available for real-time analysis. The bi-perspective approach would overcome the disadvantages of the single-method XAI systems that provide either too abstract or too simple explanations. Additionally, the integration of XAI is in line with the new regulatory obligations that mandate transparency of automated decision-making systems, including in areas like cybersecurity [18].

In comparison, the proposed system provides analysts with insights into model predictions through SHAP and LIME explanations, potentially supporting more informed decision-making [18]. The combination of SHAP (global, game-theoretic attributions) and LIME (local, surrogate-based explanations) provide multiple views of model behavior that can improve the analyst's understanding of model reasoning [35]. The combined use of SHAP and LIME provides complementary perspectives on model predictions. SHAP offers globally consistent feature attributions based on Shapley values, suitable for auditing and compliance. LIME offers localized, intuitive explanations accessible for real-time analysis. This dual-perspective approach addresses limitations of single-method XAI systems, which may provide either overly abstract or overly simplistic explanations [30].

To contextualize the contributions of this work, it is essential to compare the proposed framework against recent state-of-the-art approaches in explainable intrusion detection. As presented in Table 11, the proposed CNN-LSTM model with integrated SHAP

and LIME explainability achieves 98.7% accuracy on CICIDS2017, representing a competitive performance relative to existing methods. While Guo et al. [23] reported marginally higher accuracy (99.2%) with a Transformer-based architecture, Ferrag et al. [38] achieved 99.1% with a deep ensemble approach; both methods sacrifice critical operational requirements: neither provides real-time explainability, which is paramount for Security Operations Center deployment. In contrast, the proposed framework uniquely achieves the improved balance across three critical dimensions: (1) high detection accuracy (98.7%), (2) comprehensive dual-method explainability through SHAP and LIME, and (3) genuine real-time execution with sub-50 ms latency.

Comparative analysis reveals that existing explainable approaches either compromise on accuracy or operational efficiency. For instance, Marino et al. [21] combined deep neural networks with LIME on CICIDS2017, achieving 96.8% accuracy with partial real-time performance. The proposed framework's parallel execution strategy for SHAP and LIME, combined with INT8 quantization and LRU caching, enables explanation generation in 39–46 ms on average, representing a 73–74% latency reduction compared to sequential approaches. This operational efficiency, combined with 98.7% accuracy and comprehensive explainability, positions the framework as an operational-feasible real-time framework for threat detection and analyst decision-support in enterprise Security Operations Centers. The results demonstrate that high detection performance, interpretability, and operational efficiency are compatible objectives when properly engineered through systems-level optimizations rather than mathematical innovations.

Despite its contributions, the proposed framework has significant limitations that warrant discussion. First, it is not as effective at detecting stealthy and low-frequency attacks, such as Advanced Persistent Threats (APTs), which are not easy to detect because they tend to mimic the pattern of normal traffic and do not have any unique statistical or temporal characteristics. The limitation is due to a common problem in IDS research, which is that the anomaly detection algorithms cannot detect subtle behavioral anomalies. Second, flow-based features offer very little visibility of the encrypted traffic that is becoming commonplace in today's networks. Application-layer attacks are hard to detect without payload inspection and may need hybrid approaches that rely on metadata analysis or side-channel. Third, when the number of training samples is small for minority attack classes, the quality of the explanations gets worse because the feature attribution methods lack stability in small-density feature spaces. This is consistent with previous studies in XAI that have shown a strong relationship between model confidence and distribution of data in the explanation. The explanations of rare attacks should therefore be taken with a pinch of salt and further studies on uncertainty-aware XAI methods are encouraged. Finally, the model is prone to concept drift, meaning that it will need to be retrained periodically to ensure that it continues to perform well as the attack pattern evolves. This is a common limitation of Data-driven IDS systems, but it shows the need for adaptive or continuous learning systems that can react to new threats.

The current study focuses on two benchmark datasets: CICIDS2017 and NSL-KDD. For model robustness to extreme distribution shifts, the CICIDS2017 is used, representing the state-of-the-art attacks as of 2017, while the NSL-KDD is used as a historical benchmark. But the threat landscape is constantly changing and new attack vectors, such as attacks on IoT and new vulnerabilities [35], are not included in these datasets. Performance will need to be evaluated on more recent datasets, such as CICIDS2019 and TON_IoT, in the future [39].

Cross-dataset evaluation proves the model reasonably transfers to other traffic distributions in agreement with previous studies concerning the transferability of IDS models. But the threat landscape is dynamic and new attack vectors, especially those targeting the Internet of Things and using new vulnerabilities, are not included in these datasets.

These are just a few of the realities of implementing in the field such as heterogeneous networks, evolving threat landscapes, and privacy considerations. In this context, explainability has significant importance as a tool to build analyst trust, to validate and debug models, and for regulatory needs in the context of automated decision-making systems.

This research yields a number of interesting areas for further research. First, the model's statistical flow-based features mean that encrypted traffic and complex, application-specific anomalies are hidden from the model. There is a need for further research on hybrid approaches using metadata analysis, side-channel information or traffic analysis techniques on encrypted traffic. Second, feature attribution algorithms become less interpretable in the case of minority classes with very few examples distributed in a low-density space, because of the mathematical difficulties in estimating the decision boundaries in low-density spaces [30]. The research needs to be continued in the direction of uncertainty quantification measures, which measure the level of trust on the explanations of XAI, especially when dealing with adversarial attacks and class imbalance.

5. Conclusions

In this paper, we discussed an operationally optimized, systems-engineered approach to real-time explainable intrusion detection. Our system combines CNN-LSTM with SHAP and LIME implementations that are computationally efficient, resulting in 98.7% accuracy and 98.6% F1-score with sub-50 ms latency for generating explanations. With INT8 quantization, asynchronous parallel processing and LRU caching, explanation latency has been lowered from 117 ms (sequential) to 39 ms (average with caching) and 46 ms (without caching), or 60% improvement, which allows for real-time deployment to Security Operations Centers. Combining the global (SHAP) and local (LIME) explanations offers more than one perspective on the model predictions and increases the interpretability while not impacting model speed. The system is superior to the traditional machine learning and deep learning baselines on two popular datasets: CICIDS2017 and NSL-KDD.

Although these contributions have been made, there are a number of challenges. It needs to be investigated further in detecting stealthy and low-frequency attacks, encrypted traffic, and performance under concept drift. Future studies ought to examine adaptive learning approaches, adversarial robustness, and semantically meaningful explanation models, which align model outputs with higher-level attack knowledge. Furthermore, using new data sets (such as CICIDS2019) and traffic specific to the IoT (such as TON_IoT) to prove its effectiveness against new topologies of threats.

To sum up, the work is an important step to the creation of reliable, explainable, and scalable intrusion detection systems, which can help close the divide between the advanced AI potential and the practical cybersecurity requirements.

Author Contributions: Conceptualization, A.S., J.Z. and A.A.; methodology, A.S., J.Z. and A.A.; software, A.S., J.Z. and A.A.; validation, A.S., J.Z. and A.A.; formal analysis, J.Z.; investigation, A.S. and A.A.; resources, A.S. and A.A.; data curation, A.S. and A.A.; writing—original draft preparation, A.S., J.Z. and A.A.; writing—review and editing, A.S., J.Z. and A.A.; visualization, A.S., J.Z. and A.A.; supervision, A.S., J.Z. and A.A.; project administration, A.S., J.Z. and A.A.; funding acquisition, A.S., J.Z. and A.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are publicly accessible in the Canadian Institute of Cybersecurity Intrusion Detection System (CICIDS2017) and NSL-KDD (Network

Security Laboratory—KDD) at the URLs: https://www.unb.ca/cic/datasets/ids-2017.html, accessed on 15 January 2026.

Acknowledgments: We acknowledge the great collaboration between The Al-Ahliyya Amman University and the University of Petra.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Braue, D. Cybercrime to Cost the World $12.2 Trillion Annually By 2031. *Cybercrime Magazine*, 28 May 2025.
2. Ferrag, M.A.; Maglaras, L.; Moschoyiannis, S.; Janicke, H. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *J. Inf. Secur. Appl.* **2020**, *50*, 102419. https://doi.org/10.1016/j.jisa.2019.102419.
3. Apruzzese, G.; Colajanni, M.; Ferretti, L.; Guido, A.; Marchetti, M. On the Effectiveness of Machine and Deep Learning for Cyber Security. In Proceedings of the 2018 10th International Conference on Cyber Conflict (CyCon), Tallinn, Estonia, 29 May–1 June 2018; pp. 371–390. https://doi.org/10.23919/CYCON.2018.8405026.
4. Shalaldeh, A.; Safa, M.; Logan, C.; Othman, M. Estimation of Ewe Live Weight and Carcass Traits Using Advanced Hybrid Deep Learning and Multimodal Feature Fusion. *Biology* **2026**, *15*, 815. https://doi.org/10.3390/biology15100815.
5. Halbouni, A.; Gunawan, T.S.; Habaebi, M.H.; Halbouni, M.; Kartiwi, M.; Ahmad, R. CNN-LSTM: Hybrid Deep Neural Network for Network Intrusion Detection System. *IEEE Access* **2022**, *10*, 99837–99849. https://doi.org/10.1109/ACCESS.2022.3206425.
6. Sommer, R.; Paxson, V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In Proceedings of the 2010 IEEE Symposium on Security and Privacy, Oakland, CA, USA, 16–19 May 2010; pp. 305–316. https://doi.org/10.1109/SP.2010.25.
7. Almomani, O.; Arabiat, A.M.; Hind, A.-A.; Alsariera, E. Hybridization of Deep Learning Models for Multiclass Attack Detection in Wireless Sensor Networks. *J. Commun.* **2026**, *21*, 20–34. https://doi.org/10.12720/jcm.21.1.20-34.
8. Kılıçarslan, S.; Adem, K.; Çelik, M. An overview of the activation functions used in deep learning algorithms. *J. New Results Sci.* **2021**, *10*, 75–88. https://doi.org/10.54187/jnrs.1011739.
9. Şahin, E.; Arslan, N.N.; Özdemir, D. Unlocking the black box: An in-depth review on interpretability, explainability, and reliability in deep learning. *Neural Comput. Appl.* **2024**, *37*, 859–965. https://doi.org/10.1007/s00521-024-10437-2.
10. Abualhaj, M.M.; Al-Mimi, H.; Al-Zyoud, M.; Al-Khatib, S.N.; Daoud, M.S.; Al-Aqrabi, H.; Anbar, M.; Shalaldeh, A. A bio inspired hybrid optimization framework for efficient real time malware detection. *Sci. Rep.* **2026**, *16*, 4542. https://doi.org/10.1038/s41598-025-33439-z.
11. Hyassat, A.S.; Abu Zayed, R.E.; Al Khateeb, E.A.; Shalaldeh, A.; Abdelhamied, M.M.; Qaddara, I. Unveiling Cyber Threats: An In-Depth Study on Data Mining Techniques for Exploit Attack Detection. *Eng. Proc.* **2025**, *104*, 28. https://doi.org/10.3390/engproc2025104028.
12. Wang, W.; Sheng, Y.; Wang, J.; Zeng, X.; Ye, X.; Huang, Y.; Zhu, M. HAST-IDS: Learning Hierarchical Spatial-Temporal Features Using Deep Neural Networks to Improve Intrusion Detection. *IEEE Access* **2018**, *6*, 1792–1806. https://doi.org/10.1109/ACCESS.2017.2780250.
13. Krichen, M.; Mihoub, A. Long Short-Term Memory Networks: A Comprehensive Survey. *AI* **2025**, *6*, 215. https://doi.org/10.3390/ai6090215.
14. Bamber, S.S.; Katkuri, A.V.R.; Sharma, S.; Angurala, M. A hybrid CNN-LSTM approach for intelligent cyber intrusion detection system. *Comput. Secur.* **2025**, *148*, 104146. https://doi.org/10.1016/j.cose.2024.104146.
15. Meng, C.; Trinh, L.; Xu, N.; Enouen, J.; Liu, Y. Interpretability and fairness evaluation of deep learning models on MIMIC-IV dataset. *Sci. Rep.* **2022**, *12*, 7166. https://doi.org/10.1038/s41598-022-11012-2.
16. Qamar, T.; Bawany, N.Z. Understanding the black-box: Towards interpretable and reliable deep learning models. *PeerJ Comput. Sci.* **2023**, *9*, e1629. https://doi.org/10.7717/peerj-cs.1629.
17. Ali, S.; Abuhmed, T.; El-Sappagh, S.; Muhammad, K.; Alonso-Moral, J.M.; Confalonieri, R.; Guidotti, R.; Del Ser, J.; Díaz-Rodríguez, N.; Herrera, F. Explainable Artificial Intelligence (XAI): What we know and what is left to attain Trustworthy Artificial Intelligence. *Inf. Fusion* **2023**, *99*, 101805. https://doi.org/10.1016/j.inffus.2023.101805.
18. Hermosilla, P.; Berríos, S.; Allende-Cid, H. Explainable AI for Forensic Analysis: A Comparative Study of SHAP and LIME in Intrusion Detection Models. *Appl. Sci.* **2025**, *15*, 7329. https://doi.org/10.3390/app15137329.
19. Aysel, H.I.; Cai, X.; Prugel-Bennett, A. Explainable Artificial Intelligence: Advancements and Limitations. *Appl. Sci.* **2025**, *15*, 7261. https://doi.org/10.3390/app15137261.

20. Zhang, C.; Li, J.; Wang, N.; Zhang, D. Research on Intrusion Detection Method Based on Transformer and CNN-BiLSTM in Internet of Things. *Sensors* **2025**, *25*, 2725. https://doi.org/10.3390/s25092725.
21. Marino, D.L.; Wickramasinghe, C.S.; Manic, M. An Adversarial Approach for Explainable AI in Intrusion Detection Systems. In Proceedings of the IECON 2018—44th Annual Conference of the IEEE Industrial Electronics Society, Washington, DC, USA, 30 December 2018. https://doi.org/10.1109/IECON.2018.8591457.
22. Turbé, H.; Bjelogrlic, M.; Lovis, C.; Mengaldo, G. Evaluation of post-hoc interpretability methods in time-series classification. *Nat. Mach. Intell.* **2023**, *5*, 250–260. https://doi.org/10.1038/s42256-023-00620-w.
23. Guo, C.; Ping, Y.; Liu, N.; Luo, S.-S. A two-level hybrid approach for intrusion detection. *Neurocomputing* **2016**, *214*, 391–400. https://doi.org/10.1016/j.neucom.2016.06.021.
24. Ekpo, O.; Casola, V.; De Benedictis, A.; Asuquo, P.; Agbor, B. A Hybrid CNN–LSTM–Attention Framework for Intrusion Detection in Smart Mobility Networks. *Future Internet* **2026**, *18*, 210. https://doi.org/10.3390/fi18040210.
25. Ahmed, U.; Nazir, M.; Sarwar, A.; Ali, T.; Aggoune, E.-H.M.; Shahzad, T.; Khan, M.A. Signature-based intrusion detection using machine learning and deep learning approaches empowered with fuzzy clustering. *Sci. Rep.* **2025**, *15*, 1726. https://doi.org/10.1038/s41598-025-85866-7.
26. Genuario, F.; Santoro, G.; Giliberti, M.; Bello, S.; Zazzera, E.; Impedovo, D. Machine Learning-Based Methodologies for Cyber-Attacks and Network Traffic Monitoring: A Review and Insights. *Information* **2024**, *15*, 741. https://doi.org/10.3390/info15110741.
27. Nedeljkovic, D.; Jakovljevic, Z. CNN based method for the development of cyber-attacks detection algorithms in industrial control systems. *Comput. Secur.* **2022**, *114*, 102585. https://doi.org/10.1016/j.cose.2021.102585.
28. Yu, Y.; Long, J.; Cai, Z. Session-Based Network Intrusion Detection Using a Deep Learning Architecture. In *Modeling Decisions for Artificial Intelligence*; Springer International Publishing: Cham, Switzerland; 2017.
29. Kasongo, S.M. A deep learning technique for intrusion detection system using a Recurrent Neural Networks based framework. *Comput. Commun.* **2023**, *199*, 113–125. https://doi.org/10.1016/j.comcom.2022.12.010.
30. Aksholak, G.; Bedelbayev, A.; Magazov, R.; Kaplan, K. Transformer Tokenization Strategies for Network Intrusion Detection: Addressing Class Imbalance Through Architecture Optimization. *Computers* **2026**, *15*, 75. https://doi.org/10.3390/computers15020075.
31. Kuppa, A.; Le-Khac, N.-A. Adversarial XAI Methods in Cybersecurity. *IEEE Trans. Inf. Forensics Secur.* **2021**, *16*, 4924–4938. https://doi.org/10.1109/TIFS.2021.3117075.
32. Islam, S.R.; Eberle, W.; Ghafoor, S.K.; Ahmed, M. Explainable Artificial Intelligence Approaches: A Survey. *arXiv* **2021**, arXiv:2101.09429.
33. Vinayakumar, R.; Alazab, M.; Soman, K.P.; Poornachandran, P.; Al-Nemrat, A.; Venkatraman, S. Deep Learning Approach for Intelligent Intrusion Detection System. *IEEE Access* **2019**, *7*, 41525–41550. https://doi.org/10.1109/ACCESS.2019.2895334.
34. Gaspar, D.; Silva, P.; Silva, C. Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron. *IEEE Access* **2024**, *12*, 30164–30175. https://doi.org/10.1109/ACCESS.2024.3368377.
35. Mahbooba, B.; Timilsina, M.; Sahal, R.; Serrano, M. Explainable Artificial Intelligence (XAI) to Enhance Trust Management in Intrusion Detection Systems Using Decision Tree Model. *Complexity* **2021**, *2021*, 6634811. https://doi.org/10.1155/2021/6634811.
36. Kalutharage, C.S.; Liu, X.; Chrysoulas, C. Neurosymbolic learning and domain knowledge-driven explainable AI for enhanced IoT network attack detection and response. *Comput. Secur.* **2025**, *151*, 104318. https://doi.org/10.1016/j.cose.2025.104318.
37. Bagheri, A.; Ebrahimi, A.; Pourkargar, D.B. Self-attention transformer architectures for cyberattack detection and secure state reconstruction of integrated chemical process systems. *Chem. Eng. Res. Des.* **2025**, *220*, 162–179. https://doi.org/10.1016/j.cherd.2025.06.038.
38. Ferrag, M.A.; Maglaras, L.; Ahmim, A.; Derdour, M.; Janicke, H. RDTIDS: Rules and Decision Tree-Based Intrusion Detection System for Internet-of-Things Networks. *Future Internet* **2020**, *12*, 44. https://doi.org/10.3390/fi12030044.
39. Villafranca, A.; Thant, K.M.; Tasic, I.; Cano, M.-D. AI-Enabled IoT Intrusion Detection: Unified Conceptual Framework and Research Roadmap. *Mach. Learn. Knowl. Extr.* **2025**, *7*, 115. https://doi.org/10.3390/make7040115.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.